

Fundamentos de Programación.

Guion de Prácticas.

Curso 2026/2027

© CopyRight: Juan Carlos Cubero. Universidad de Granada.
Sugerencias: por favor, envíe un e-mail a
JC.Cubero@decsai.ugr.es

"Lo que tenemos que aprender a hacer, lo aprendemos haciéndolo".
Aristóteles

"In theory, there is no difference between theory and practice. But, in practice, there is".
Jan L. A. van de Snepscheut

"The gap between theory and practice is not as wide in theory as it is in practice".

"Theory is when you know something, but it doesn't work. Practice is when something works, but you don't know why. Programmers combine theory and practice: Nothing works and they don't know why".



Sobre el guion de prácticas

Este apartado explica las directrices generales para las clases prácticas.

- El guion está dividido en sesiones. En cada sesión se plantean una serie de problemas de programación a resolver. En la semana número i se publicará la **Sesión** i. En dicha sesión se especifica la lista de problemas que el alumno tiene que resolver.

- Las soluciones de los ejercicios deberán ser subidas a la plataforma PRADO . El alumno subirá un fichero `zip` que contendrá los ficheros con extensión `cpp` correspondientes a las soluciones de los ejercicios (no incluya los ejecutables .exe).

  El fichero `zip` deberá nombrarse con el nombre y apellidos del alumno (sin espacios en blanco, acentos, etc) Por ejemplo, si su nombre es Pedro Magaña García, nómbrelo como `MaganiaGarciaPedro.zip`

  Nombre los ficheros `cpp` según sea el ejercicio pero no use acentos ni espacios en blanco. Añádale al final el número del ejercicio. Por ejemplo: `pi_arcoseno_4.cpp`.

  Incluya su nombre y apellidos como comentario al principio de cada uno de los ficheros `cpp`.

- La defensa de la sesión i se hará la semana siguiente (semana $i+1$), durante las horas de prácticas. El profesor llamará aleatoriamente a los alumnos para que defiendan dichos ejercicios. La defensa podrá consistir en la realización en papel de alguno de los ejercicios de la sesión o explicarlo a los compañeros.

  Muy importante:

  - *La resolución de los problemas y actividades puede hacerse en grupo, pero la defensa durante las horas de prácticas es individual.*

  - Se recomienda que las herramientas de **Inteligencia Artificial** las utilice para resolver dudas (al igual que el trabajo en grupo) y no para que resuelvan ellas completamente los problemas planteados. Dichas herramientas no estarán disponibles en las defensas ni en ninguno de los distintos exámenes de la asignatura.

  Simultáneamente a la defensa, todos los alumnos tendrán que ir realizando una serie de actividades que vienen descritas en este guion (bajo el epígrafe *Actividades a realizar en las aulas de ordenadores*). *Dichas actividades no hay que entregarlas al profesor*.

  Terminada la defensa, el profesor debatirá con los alumnos las soluciones de los ejercicios.

La organización por semanas es la siguiente:

- del 14 al 18 de Septiembre: Empiezan las clases de prácticas en las aulas de ordenadores. Durante las dos horas de la sesión de prácticas, los alumnos realizarán las actividades descritas en la sesión 1 (ver página 6). Consulte los horarios de las clases prácticas

en el siguiente enlace:

`https://etsiit.ugr.es/docencia/grados`

Durante esta semana, los alumnos deben resolver en su casa los ejercicios indicados en la sesión 1 (ver página 16). Las soluciones a dichos ejercicios deben entregarse en PRADO antes de las 7:00h del Lunes 21 de Septiembre.

- del 21 de Septiembre al 25 de Septiembre: Durante las clases de prácticas, los alumnos defenderán los ejercicios de la sesión 1 que previamente han entregado. Además, en sus casas, los alumnos deben resolver los ejercicios indicados en la sesión 2 (ver página 18). Las soluciones a dichos ejercicios deben entregarse en PRADO antes de las 7:00h del Lunes 28 de Septiembre.
- El resto de semanas se organizan de forma análoga a lo indicado en el punto anterior.

Los problemas a resolver en cada sesión están incluidos en las *Relaciones de Problemas*. Hay una relación de problemas por cada tema de la asignatura. Los ejercicios de cada sesión son de varios tipos:

- *Lectura de programas resueltos*: Son ejercicios resueltos que el alumno debe leer y entender. En algunas sesiones tendrá que usar alguna de las técnicas descritas en dichos ejercicios para poder resolver alguno de los ejercicios obligatorios.
- *Obligatorios*: Todos los alumnos deben resolver y entregar estos problemas.
- *Opcionales*: Son ejercicios algo más complejos cuya entrega no es obligatoria. En cualquier caso, es muy recomendable que los alumnos intenten resolverlos para que pongan a prueba su evolución como programadores.
- *Complementarios*: No han de entregarse. Son ejercicios sencillos que pueden servir de refuerzo para aquellos alumnos que están empezando a programar.
- *Complementarios avanzados*: No han de entregarse. Son ejercicios típicos de un examen, por lo que se recomienda que el alumno intente resorverlos.

Además de los ejercicios anteriores, en las relaciones de problemas también encontrará *ejercicios adicionales* por si quiere practicar. Dichos ejercicios no se verán en clase y tampoco ha de entregar ninguno de ellos.

Para la realización de estas prácticas, se utilizará el entorno de programación Visual Studio Code. En la página 4 se encuentran las instrucciones para su instalación en nuestra casa. En cualquier caso, el alumno puede instalar en su casa cualquier otro compilador como por ejemplo Embarcadero DevC++ o CodeBlocks. El día del examen de prácticas sólo tendrá disponible los tres anteriores entornos en las aulas de ordenadores.

Instalación de VSCode y de winscp en nuestra casa

- El entorno de desarrollo que usaremos será Visual Studio Code. Para la instalación en su casa, siga las indicaciones del siguiente enlace:
  `https://decsai.ugr.es/jccubero/FP/InstalarVSCode.pdf`
- Cada alumno tendrá disponible durante toda la carrera un espacio de almacenamiento en los servidores de la Escuela. Para acceder a dicho espacio, basta con seleccionar la unidad `U:` si está en un ordenador de la Escuela. Desde nuestras casas, debemos usar cualquier programa de `ftp` que use el protocolo `ssh`, como por ejemplo `filezilla` o `winscp`. Instalamos este programa en nuestra casa y simplemente nos conectamos a `turing.ugr.es` con nuestras credenciales, es decir, usando el mismo nombre de usuario y contraseña con el que iniciamos sesión en las clases de prácticas (ver página 16).

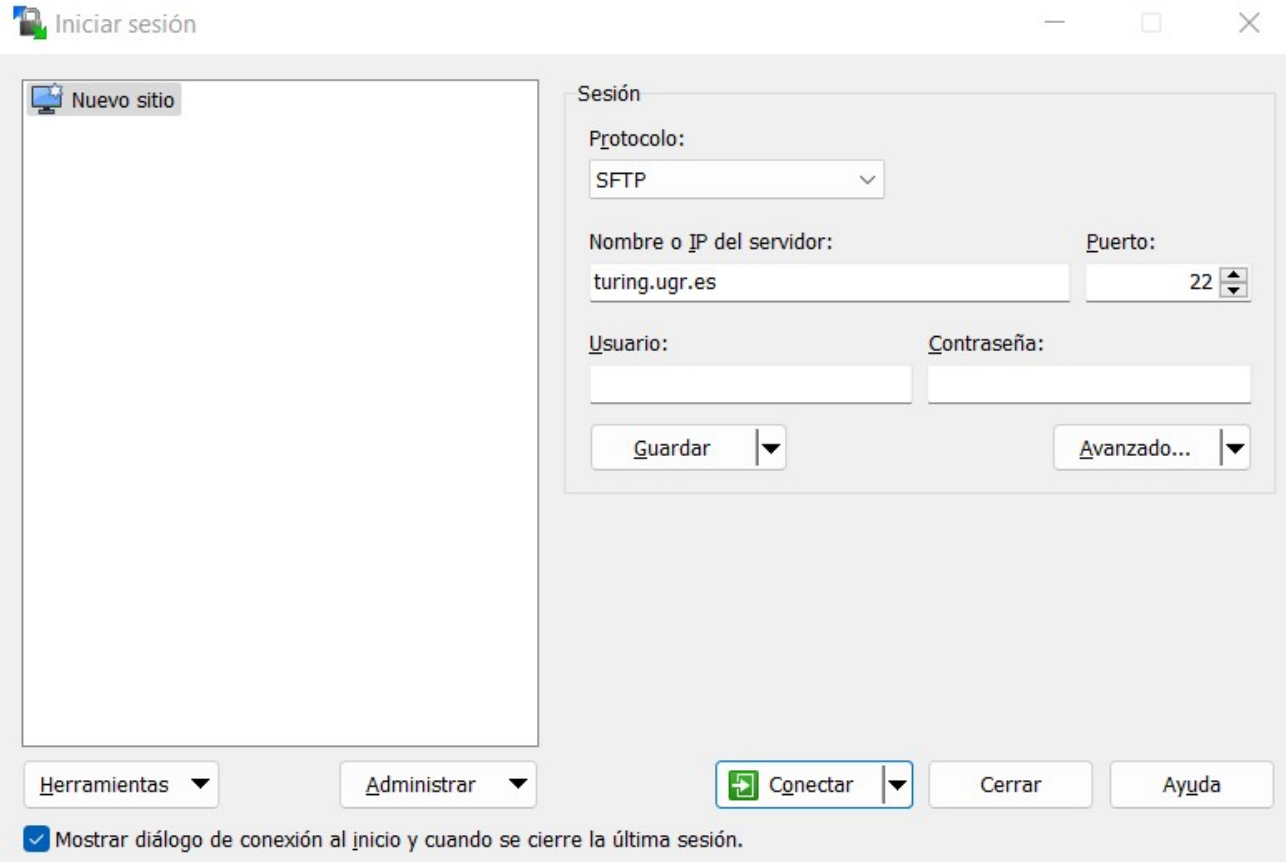


Figura 1: Conexión a la unidad U: usando winscp

Tabla resumen de accesos directos usados en VSCode

Tab	Tabula una línea o un bloque
Shift Tab	Quita tabulación a una línea o un bloque
Ctr F	Buscar
Ctr H	Buscar y reemplazar
Ctrl Barra Espaciadora	Ayuda autocompletación del código
Shift Ctrl Barra Espaciadora	Ayuda parámetros funciones y métodos
F5	Ejecutar con depuración
Ctr F5	Ejecutar sin depuración
F10	Ejecución paso a paso sin entrar en los métodos o funciones
F11	Ejecución paso a paso entrando en los métodos o funciones
Shift F11	Sale de la ejecución del método o función

Sesión 1 (Tema I)

Esta sesión consta de:

1. *Actividades a realizar en las aulas de ordenadores*. Son las actividades que se realizarán con la ayuda del profesor durante las clases de prácticas de la semana del 14 al 18 de Septiembre.
2. *Actividades a realizar en casa.* Son las tareas que el alumno debe realizar en su casa durante la semana del 14 al 18 de Septiembre. Las soluciones a los ejercicios indicados en dicha sesión deben entregarse en PRADO antes de las 7:00h del Lunes 21 de Septiembre.

► Actividades a realizar en las aulas de ordenadores

Estas son las actividades que se realizarán, con la ayuda del profesor, en las aulas de ordenadores durante la semana del 14 al 18 de Septiembre.

El Entorno de Programación. Compilación de Programas

Arranque del Sistema Operativo

Para poder arrancar el SO en las aulas de ordenadores, debe introducir como usuario el texto que hay antes de `@` en su correo electrónico de la UGR. Por ejemplo, si su correo es `aperez@correo.ugr.es` tendrá que introducir `aperez`.

En la casilla etiquetada como `Código`, introduciremos el código proporcionado por el profesor de prácticas ese mismo día. Al arrancar el SO, aparecerá una instalación básica de Windows con el entorno Visual Studio Code. Todo lo que escribamos en la unidad `C:` se perderá al apagar el ordenador.

Por ello, tal y como se indicó en la página 4 el alumno dispone de un espacio de almacenamiento en la unidad lógica `U:`, cuyos contenidos permanecerán durante todo el curso académico. El alumno deberá crear el directorio `U:\FP` dentro de su unidad `U:`. El nombre debe ser exactamente el anterior, todo en mayúsculas.

En cualquier caso, es recomendable no saturar el espacio usado ya que, en caso contrario, el compilador podría no funcionar. Recuerde que las entregas de las prácticas se realizarán siempre a través de PRADO . La unidad `U:` es una unidad de almacenamiento disponible para el alumno.

El primer programa

Copiando el código fuente

Cree la carpeta `Tema_1` colgando de `U:\FP:` Descargue ahora el fichero
`https://decsai.ugr.es/jccubero/FP/I_Pitagoras.cpp`
y cópielo dentro de la carpeta `Tema_1`.
Abra el entorno VSCode y desde la pestaña del Explorador navegue por las carpetas para buscar el fichero `I_Pitagoras.cpp` y haga doble click sobre él. Hagalo siempre de esta forma (abriendo primero VSCode y seleccionando el fichero) y no haciendo doble click desde el explorador del propio Sistema Operativo. Si lo hace de la segunda forma, no se aplicarán las opciones de configuración del entorno. Ver figura 2

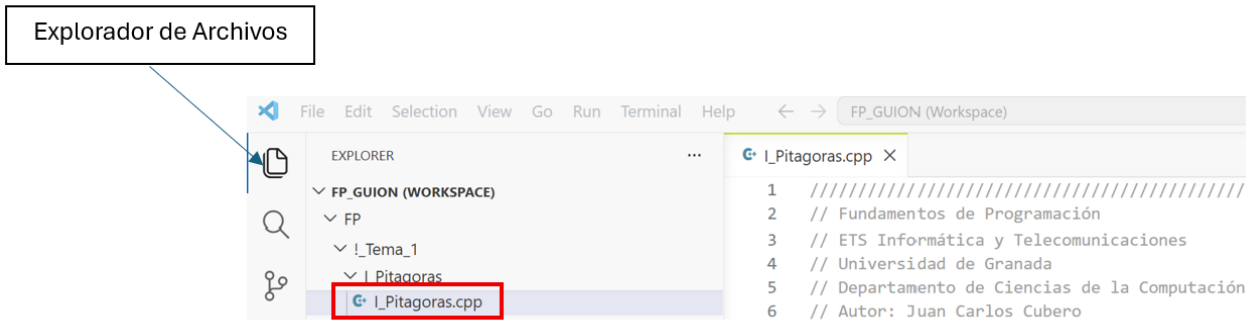


Figura 2: Abrir un fichero con código fuente

Algunas consideraciones con respecto a la escritura de código (ver figura 3)

- Los programas deben incluir comentarios:
  - Comentarios que describan la idea general del algoritmo: usualmente serán comentarios de bloque (`/* ... */`). No se debe comentar aquello que sea obvio.
  - Comentarios para explicar alguna línea concreta de código: usualmente serán comentarios de línea (`// ...`) y no debemos abusar de ellos.

  Consulte los apuntes de clase para más información.
- Es muy importante una correcta tabulación de los programas. Por ahora, incluiremos todas las sentencias del programa principal con una tabulación.
- Para facilitar la lectura del código fuente, se deben usar espacios en blanco para separar las variables en la línea en la que van declaradas, así como antes y después del símbolo

```
#include <iostream>   // Inclusión de recursos de E/S
#include <cmath>      // Inclusión de recursos matemáticos
using namespace std;

int main(){                 // Programa Principal
   double lado1;            // Declara variables para guardar
   double lado2;            // los dos lados y la hipotenusa
   double hipotenusa;

   cout << "Introduzca la longitud del primer cateto: ";
   cin >> lado1;
   cout << "Introduzca la longitud del segundo cateto: ";
   cin >> lado2;

   hipotenusa = sqrt(lado1*lado1 + lado2*lado2);

   cout << "\nLa hipotenusa vale " << hipotenusa;
```

Todas las sentencias dentro de un bloque { } van tabuladas.

Espacio en blanco para facilitar la lectura de cada sentencia

Línea en blanco entre sentencias para separar bloques de código

Figura 3: Escritura de código

= en una sentencia de asignación. Deje también un espacio en blanco antes y después de << y >> en las sentencias que contienen una llamada a `cout` y `cin` respectivamente.

No respetar las normas de escritura de código baja puntos en todos los exámenes y prácticas de la asignatura



Compilación

Una vez cargado el programa, pasamos a comprobar si las sentencias escritas son sintácticamente correctas, es decir, pasamos a *compilar* el programa. Para ello pulsamos `Shift-Ctr-B` o desde el menú de VSCode seleccionamos `Terminal/Run Build Task`.

Para que el proceso de compilación se realice de forma correcta y se obtenga el programa ejecutable, es necesario que el código fuente no contenga errores sintácticos. Si aparecen errores, es necesario volver a la fase de edición, guardar de nuevo el código fuente y repetir la fase de compilación.

Como resultado de la fase de compilación, en la parte de abajo del entorno debe aparecer un mensaje del tipo `Build finished successfully`



Figura 4: Fase de compilación

Una vez compilado el programa, habremos obtenido el fichero `I_Pitagoras.exe`, directamente ejecutable desde el sistema operativo. Este fichero se encuentra en el mismo di-

rectorio que el del fichero `cpp`. Para mostrar que el fichero generado es independiente del entorno de programación, haga lo siguiente:

1. Cierre VSCode.
2. Abra una ventana de Mi PC.
3. Sitúese en la carpeta que contiene el ejecutable.
4. Haga doble click sobre el fichero `I_Pitagoras.exe`.

El programa terminará cuando se ejecute la última instrucción. El proceso del sistema operativo asociado al programa terminará y se cerrará la ventana correspondiente sin dar tiempo a ver la salida de resultados. Desde VSCode no hay problema ya que éste mantiene abierta la terminal para ver los resultados. En el caso de que quiséramos hacer una pausa desde el programa ejecutado desde el sistema operativo, habría que leer un dato cualquiera con `cin` o usar otro recurso del propio sistema operativo (no lo haremos)

Para ejecutar el programa desde VSCode basta pulsar `Ctr-F5` (debe estar activa la ventana del fichero con el código fuente). Esta combinación de teclas compila y ejecuta, algo que haremos normalmente durante el desarrollo de nuestros programas. VSCode abrirá una nueva terminal para trabajar con el proceso asociado al ejecutable que hemos creado (ver figura 5). Observe que ahora tenemos dos terminales abiertas: una asociada a los resultados de la compilación y otra asociada a la ejecución del programa. La ejecución del programa se pausará en aquellos puntos del mismo donde se requiera la interacción del usuario para poder proseguir, es decir, en la operaciones de entrada de datos a través del dispositivo estándar de entrada. En este ejemplo, sería en las dos operaciones `cin`. En el resto de los casos, la ejecución del programa continuará hasta el final. La introducción de datos mediante la sentencia `cin` se hace siempre de la misma manera; primero se introduce el valor que se desee y al terminar se pulsa la tecla RETURN.

Introduzca ahora los valores pedidos en el ejemplo de Pitágoras y compruebe la respuesta del programa.

Prueba del programa

Uno podría pensar que una vez que consigo un fichero ejecutable a partir de mi código fuente, el problema está terminado. Sin embargo esto no es así. Tras el proceso de compilado se requiere una fase de prueba. Dicha fase intenta probar que el algoritmo planteado resuelve el problema propuesto. Para llevar a cabo esta fase, es necesario ejecutar el programa y verificar que los resultados que obtiene son los esperados.

Ahora que podemos ver el resultado obtenido por el programa implementado, verifiquemos mediante el siguiente conjunto de pruebas que el programa funciona de forma correcta.

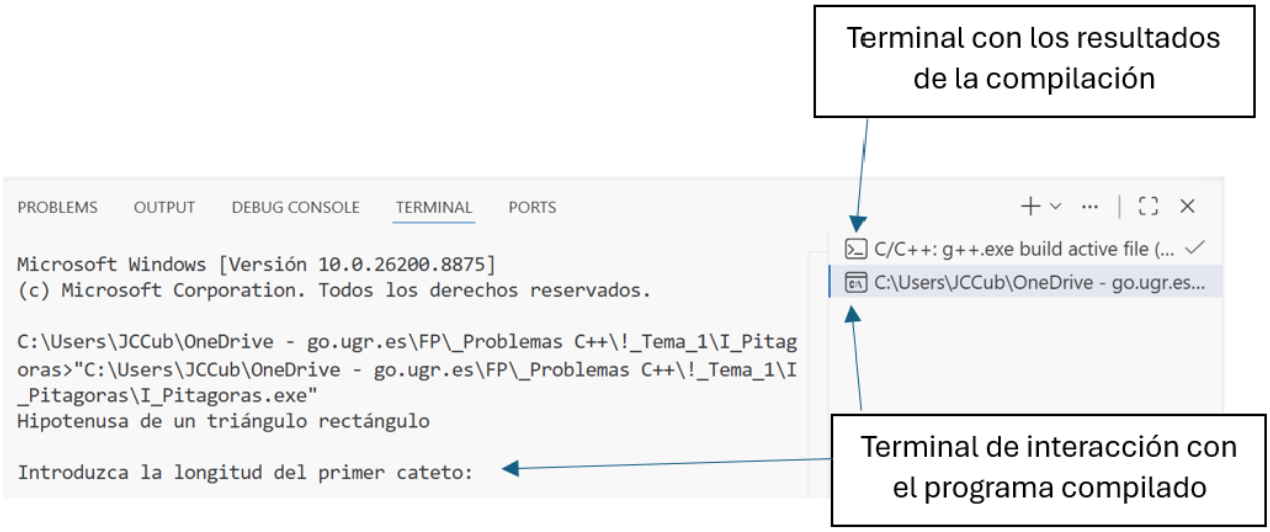


Figura 5: Fase de ejecución

lado1	lado2	hip
3	4	5
1	5	5.099
2.7	4.3	5.077
1.25	2.75	3.02

Una vez que el algoritmo supera la fase de prueba, podemos considerar que se ha concluido con la fase inicial del desarrollo del software.

Nota:

C++11 incorpora en la biblioteca `cmath` una función específica para calcular la hipotenusa, llamada `hypot`. Si lo desea, también puede comparar los resultados de su algoritmo con el ofrecido por dicha función

---

Introducción a la corrección de errores

Los errores de compilación

Ya hemos visto los pasos necesarios para construir un fichero ejecutable a partir del código fuente. El paso central de este proceso era la fase de compilación. En esta parte de este guion de prácticas aprenderemos a corregir los errores más comunes que impiden una compilación exitosa del fichero fuente.

Cargue el fichero `I_Pitagoras.cpp`. Quítele una '`u`' a alguna aparición de `cout`. Intente compilar. Podemos observar que la compilación no se ha realizado con éxito. Cuando esto sucede, en la parte inferior de la ventana principal aparecen los errores que se han

encontrado.

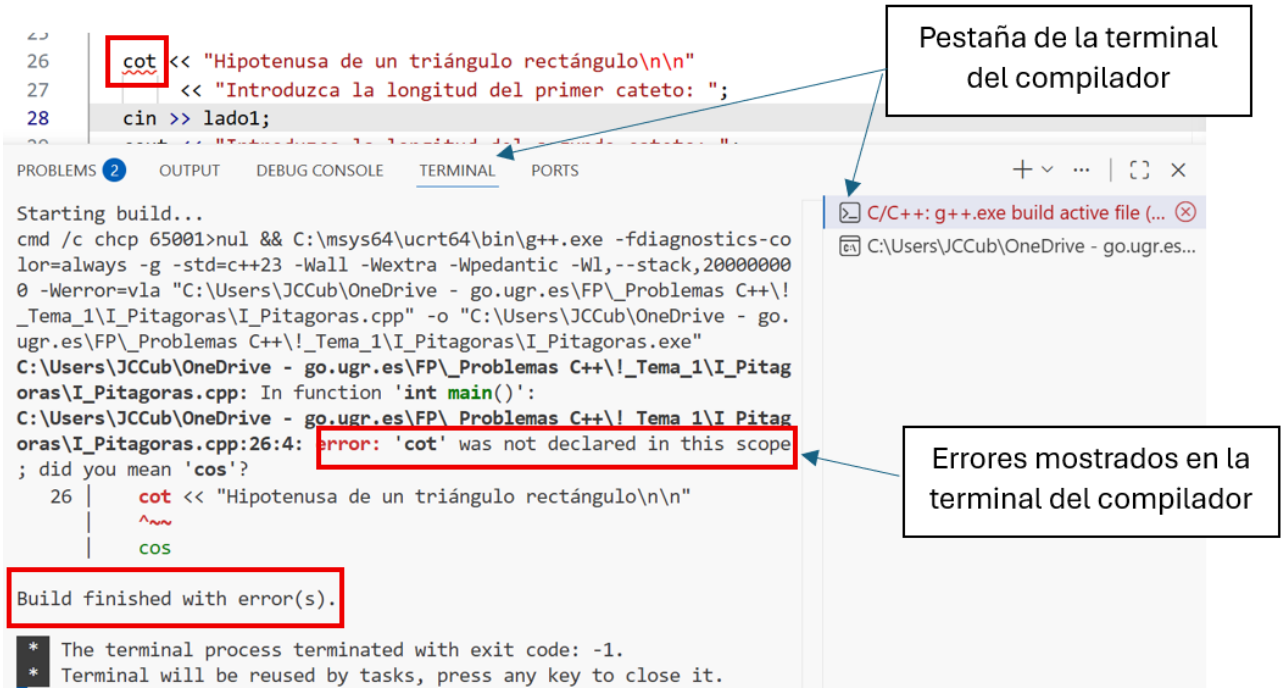


Figura 6: Errores de compilación (terminal)

El pack de C++ para VSCode ofrece la herramienta *Intellisense* que detecta los errores de compilación mientras se está escribiendo el código. Se mostrará un *gusanillo* sobre la parte del programa que presenta un error de compilación. Si mantenemos el cursor sobre el error, se mostrará una descripción del mismo. Esa información también aparece si abrimos la pestaña *Problems*: si hacemos doble click en dicha ventana sobre uno de los errores mostrados, se mostrará la línea correspondiente del código.

Los pasos que debemos seguir para la corrección de errores son los siguientes:

1. Ir a la primera fila de la lista de errores.
2. ***Leer el mensaje de error e intentar entenderlo***.
3. Hacer doble click sobre esa fila con el ratón. Esto nos posiciona sobre la línea en el fichero fuente donde el compilador detectó el error.
4. Comprobar la sintaxis de la sentencia que aparece en esa línea. Si se detecta el error, corregirlo. Si no se detecta el error mirar en la línea anterior, comprobar la sintaxis y repetir el proceso hasta encontrar el error.
5. Después de corregir el posible error, guardamos de nuevo el archivo y volvemos a com-

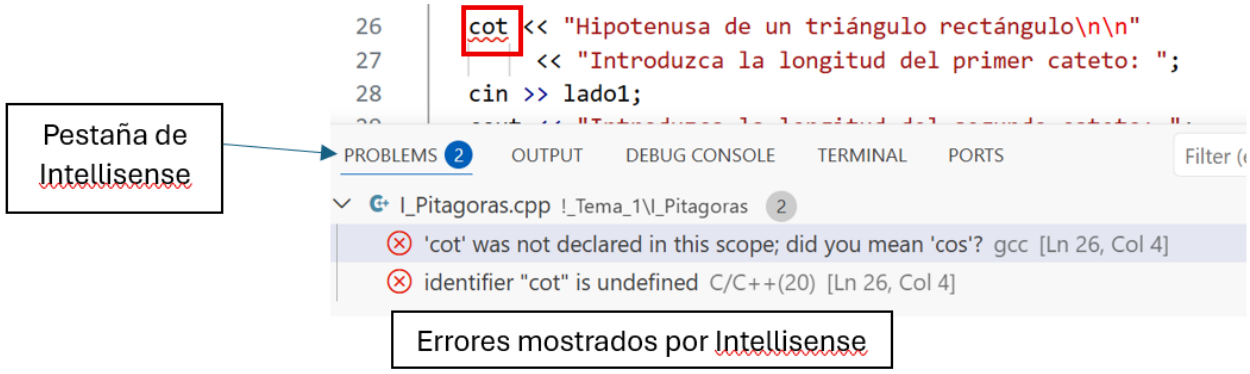


Figura 7: Errores de compilación (Intellisense)

pilar. Esto lo hacemos aunque aparezcan más errores en la ventana. La razón es que, a veces, la herramienta *Intellisense* no refresca adecuadamente las modificaciones realizadas en el editor y por tanto es necesario volver a compilar.

6. Si después de corregir el error aparecen nuevos errores, volver a repetir el proceso desde el paso 1.

A veces, el compilador no indica la línea exacta en la que se produce el error, sino alguna posterior. Para comprobarlo, haced lo siguiente:

- Comente la línea de cabecera `#include <iostream>` desde el principio. El compilador no reconocerá las apariciones de `cin` o `cout`.
- Quite un punto y coma al final de alguna sentencia. Dará el error en la línea siguiente.

Para familiarizarnos con los errores más frecuentes y su corrección vamos a realizar el siguiente proceso: a partir del código fuente del ejemplo `I_Pitagoras.cpp`, iremos introduciendo deliberadamente errores para conocer los mensajes que nos aparecen. A continuación se muestran algunos errores posibles. No deben introducirse todos ellos a la vez, sino que han de probarse por separado.

1. Cambie algún punto y coma por cualquier otro símbolo
2. Cambie `double` por `dpuble`
3. Cambie la línea `using namespace std;` por `using namespace STD;`
4. Ponga en lugar de `iostream`, el nombre `iotream`.
5. Borre alguno de los paréntesis de la declaración de la función `main`
6. Introduzca algún identificador incorrecto, como por ejemplo `cour`

7. Use una variable no declarada. Por ejemplo, en la definición de variables cambiad el nombre a la variable `lado1` por el identificador `lado11`.
8. Borre alguna de las dobles comillas en una constante de cadena de caracteres, tanto las comillas iniciales como las finales.
9. Borre alguna de las llaves que delimitan el inicio y final del programa.
10. Borre la línea `using namespace std;` (basta con comentarla con `//`)
11. Cambie un comentario iniciado con `//`, cambiando las barras anteriores por las siguientes `\\`
12. Cambie la aparición de << en `cout` por las flechas cambiadas, es decir, >>. Haced lo mismo con `cin`.
13. Suprima todo el `main`. No hace falta borrar el código, basta con comentarlo.

Además de los errores, el compilador puede generar *avisos*. Estos se muestran como `Warning` en la misma ventana *Problems*. Estas advertencias indican que algún código puede generar problemas durante la ejecución. Por ejemplo, al usar una variable que todavía no tiene un valor asignado, al intentar asignar un entero *grande* a un entero *chico*, etc. Sin embargo, no son errores de compilación, por lo que es posible generar el programa ejecutable correspondiente.

Los errores lógicos y en tiempo de ejecución

Aunque el programa compile, esto no significa que sea correcto. Puede producirse una excepción durante la ejecución, de forma que el programa terminará bruscamente (típico error en Windows de *Exception Violation Address)* o, lo que es peor, dará una salida que no es correcta (error lógico).

Sobre el programa `I_Pitagoras.cpp`, haga lo siguiente:

- Cambie la sentencia
  `sqrt(lado1*lado1 + lado2*lado2)` por:
  `sqrt(lado1*lado2 * lado2*lado2)`
  Ejecute introduciendo los lados 2 y 3. El resultado no es correcto, pero no se produce ningún error de compilación ni en ejecución. Es un error lógico.
- Para mostrar un error de ejecución, declare tres variables ENTERAS (tipo `int`) `resultado`, `numerador` y `denominador`. Asígnele cero a `denominador` y 7 a `numerador`. Asígnele a `resultado` la división de `numerador` entre `denominador`. Imprima el resultado. Al ejecutar el programa, se produce una excepción o error de ejecución al intentar dividir un entero entre cero.

Creación de un programa nuevo

En esta sección vamos a empezar a crear nuestros propios programas desde VSCode. El primer ejemplo que vamos a implementar corresponde al ejercicio I.1 [Ley de Ohm] de la relación de problemas I.

Para crear un programa nuevo, desde el explorador de VSCode creamos una carpeta nueva con el nombre `I_Voltaje`. Una vez creada la carpeta, nos situamos en ella y con el click izquierda del ratón seleccionamos *New File* y le asignamos el nombre `I_Voltaje.cpp` (ver figura 8).

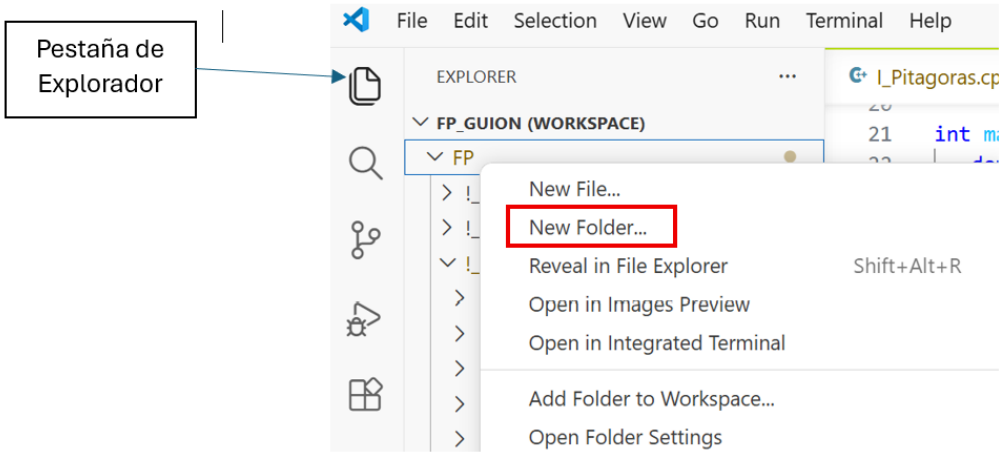


Figura 8: Creación de un programa nuevo

Ya estamos en condiciones de resolver el problema pedido. Escribimos el código en la ventana de edición. Habrá que leer desde teclado los valores de intensidad y resistencia y el programa imprimirá en pantalla el voltaje correspondiente.

Nota. Cuando tenemos varias variables en el código, podemos empezar a escribir el nombre de alguna de ellas y antes de terminar, pulsar `Ctr-Barra espaciadora`. La ayuda nos mostrará los identificadores disponibles que empiecen por las letras tecleadas.

Empiece a trabajar con las actividades a realizar en casa.

Cree un fichero `cpp` por cada uno de los ejercicios. Puede ir guardándolos en su unidad `U:` para poder acceder a ellos desde su casa (recuerde lo visto en la página 4). Estos ficheros no tiene que entregarlos en este momento. Los tendrá que entregar en PRADO para defenderlos en la semana del 21 de Septiembre al 25 de Septiembre. La fecha límite para entregarlos es las 7:00h del Lunes 21 de Septiembre.

► Actividades a realizar en casa

El alumno debe realizar estas actividades en su casa durante la semana del 14 al 18 de Septiembre:

Actividad: Conseguir login y password.

El alumno debe conseguir una cuenta de correo electrónico de la UGR, tal y como se indica en el fichero de información general de la asignatura. Las credenciales de dicha cuenta (login y password) son las mismas que habrá que usar al arrancar los ordenadores en las aulas de prácticas.

Actividad: Instalación de Visual Studio Code.

Instale en su casa el entorno Visual Studio Code. Consulte la sección de Instalación (página 4) de este guion.

Actividad: Lectura de programas resueltos.

Lea la solución de los siguientes ejercicios:

I.3 [Circunferencia]

Solución → `https://decsai.ugr.es/jccubero/FP/I_Circunferencia_sin_ctes.cpp`

Actividad: Resolución de problemas.

Resuelva los siguientes ejercicios de la Relación de Problemas I. Recuerde que debe subir a PRADO las soluciones de estos ejercicios, siguiendo las directrices indicadas en la página 2. Debe entregarlas antes de las 7:00h del Lunes 21 de Septiembre.

Se proporciona enlaces a videos que explican el esquema de la solución de algunos ejercicios. Consulte dichos videos **DESPUÉS** de haber intentado resolver el ejercicio por sus propios medios.

- *Obligatorios:*

  I.2 [Asignaciones secuenciales salario]

  I.4 [Cálculo de π a partir del arco-seno]

  Video → `https://decsai.ugr.es/jccubero/FP/I_Pi.mp4`

  I.5 [Variación porcentual]

  Video → `https://decsai.ugr.es/jccubero/FP/I_VariacPorcentual.mp4`

  I.6 [Subir sueldo]

  Video → `https://decsai.ugr.es/jccubero/FP/I_SubirSueldo.mp4`

  I.7 [Dos subidas de sueldo]

I.8 [Interés bancario]

Video → `https://decsai.ugr.es/jccubero/FP/I_InterBancario.mp4`

I.9 [Distancia Euclídea]

Video → `https://decsai.ugr.es/jccubero/FP/I_DistanciaEuclidea.mp4`

I.10 [Conversión de grados a radianes]

Video → `https://decsai.ugr.es/jccubero/FP/I_Grados_to_Radianes.mp4`

- *Opcionales:*

  I.11 [Decimal redondeado]

  Video → `https://decsai.ugr.es/jccubero/FP/I_Round.mp4`

Sesión 2 (Tema I)

► Actividades a realizar en las aulas de ordenadores

Estas actividades se realizan en el aula de prácticas durante la semana del 21 de Septiembre al 25 de Septiembre.

El profesor irá corrigiendo individualmente (a algunos alumnos elegidos aleatoriamente) los ejercicios encargados para esta semana y que el alumno ya ha tenido que entregar en PRADO . Mientras tanto, los alumnos restantes deben intentar resolver los ejercicios encargados para la próxima semana (los indicados en las *Actividades a realizar en casa* de esta sesión)

► Actividades a realizar en casa

El alumno debe realizar estas actividades en su casa durante la semana del 21 de Septiembre al 25 de Septiembre.

Actividades de Ampliación (no son obligatorias y no forman parte de la evaluación)

Recuerde los conceptos de combinación y permutación, que irán apareciendo recurrentemente a lo largo de la carrera. Consulte, por ejemplo, la siguiente web: `http://www.disfrutalasmatematicas.com/combinatoria/combinaciones-permutaciones.html`

Si por ejemplo queremos ver las posibles combinaciones (con repetición e importando el orden) de dos elementos (0 y 1) en 4 posiciones de memoria (4 bits) obtenemos un total de $2^4 = 16$ posibilidades: 0000, 0001, 0010, $\cdots$, 1111. Ejecute el siguiente applet para ver las combinaciones resultantes:

`http://dm.udc.es/elearning/Applets/Combinatoria/index.html`

Actividad: Lectura de programas resueltos.

Lea la solución de los siguientes ejercicios:

I.23 [Índice de mayúscula] (léalo después de haber visto en clase el tipo `char`)

Solución → `https://decsai.ugr.es/jccubero/FP/I_IndiceMayuscula.cpp`

Actividad: Resolución de problemas.

Recuerde que debe subir a PRADO las soluciones de estos ejercicios siguiendo las directrices indicadas en la página 2. Debe entregarlas antes de las 7:00h del Lunes 28 de Septiembre.

Se proporciona enlaces a videos que explican el esquema de la solución de algunos ejercicios. Consulte dichos videos **DESPUÉS** de haber intentado resolver el ejercicio por sus propios medios.

- *Obligatorios:*

  I.12 [Tarifa aérea según km]

  Video → `https://decsai.ugr.es/jccubero/FP/I_Tarifa_km.mp4`

  I.13 [Tarifa aérea con descuento]

  Video → `https://decsai.ugr.es/jccubero/FP/I_TarifaDescuento.mp4`

  I.14 [Segundos entre dos instantes]

  Video → `https://decsai.ugr.es/jccubero/FP/I_Instantes.mp4`

  I.16 [Intercambiar variables]

  Video → `https://decsai.ugr.es/jccubero/FP/I_Intercambiar.mp4`

  I.18 [Gaussiana]

  Video → `https://decsai.ugr.es/jccubero/FP/I_Gaussiana.mp4`

  I.19 [Intercambiar tres variables]

  Video → `https://decsai.ugr.es/jccubero/FP/I_Intercambiar3.mp4`

  I.22 [Gasolina]

  I.26 [Pasar de mayúscula a minúscula] Se recomienda que lea antes la solución al ejercicio I.23 [Índice de mayúscula] disponible en la lista de ejercicios a leer de esta sesión.

  Video → `https://decsai.ugr.es/jccubero/FP/I_ToMinuscula.mp4`

  I.28 [Escalado]

- *Opcionales:*

  I.31 [Precisión y desbordamiento]

  Video → `https://decsai.ugr.es/jccubero/FP/I_PrecisionDesbordamiento.mp4`

- *Complementarios:*

  Los siguientes ejercicios no tiene que entregarlos ni se defenderán. Si está empezando a programar le podrán ayudar a afianzar conceptos básicos.

  I.15 [Uso de constantes]

  I.17 [Sistema métrico]

  Solución → `https://decsai.ugr.es/jccubero/FP/I_ConversionSistemaMetrico.cpp`

I.20 [Media y desviación]

Solución → `https://decsai.ugr.es/jccubero/FP/I_Media.cpp`

I.21 [Aproximación del valor de π]

Solución → `https://decsai.ugr.es/jccubero/FP/I_PiAproximacion.cpp`

Sesión 3 (Temas I y II)

► Actividades a realizar en las aulas de ordenadores

Estas actividades se realizan en el aula de prácticas durante la semana del 28 de Septiembre al 2 de Octubre.

El buffer de entrada

Para comprobar el correcto funcionamiento de nuestros programas, tendremos que ejecutarlos en repetidas ocasiones usando distintos valores de entrada. Este proceso lo repetiremos hasta que no detectemos fallos. Vamos a ver cómo simplificar este proceso.

Cada vez que se ejecuta `cin`, se lee un dato desde el periférico por defecto. Si en la ejecución de una sentencia `cin` introducimos varios valores separados por espacios en blanco, todos ellos se almacenan dentro de una zona de memoria intermedia denominada buffer. Este buffer canaliza el flujo de datos entre el programa y el dispositivo de entrada de datos, de forma que las sentencias `cin` posteriores leerán los datos que hay en el buffer. Para más detalle, consulte el final del Tema 1, disponible en PRADO .

Esto nos permite copiar todos los datos de entrada que mi programa necesita (separados por espacios en blanco) y pegarlos en la ejecución de la primera sentencia `cin`. Las ejecuciones posteriores de las sentencias `cin` irán leyendo los datos que hay almacenados en el buffer. Cuando el buffer esté vacío, automáticamente se solicitarán más datos al dispositivo de entrada de datos.

Para comprobarlo, copie localmente el fichero `https://decsai.ugr.es/jccubero/FP/II_cin.cpp` . El programa simplemente lee un entero y dos caracteres y los imprime en pantalla. Cree un fichero de texto con un entero y dos caracteres. Separe estos tres datos con varios espacios en blanco. Seleccione con el ratón los tres y cópielos al portapapeles (Click derecho-Copiar). Ejecute el programa y cuando aparezca la consola del sistema haga click derecho sobre la ventana y seleccione `Editar-Pegar`. También puede usar la típica combinación de teclas `Ctrl-C` para copiar y `Ctrl-V` para pegar.

Empiece a trabajar con las actividades a realizar en casa.

► Actividades a realizar en casa

El alumno debe realizar estas actividades en su casa durante la semana del 28 de Septiembre al 2 de Octubre.

Actividad: Lectura de programas resueltos.

Lea la solución de los siguientes ejercicios:

Ejercicios de la Relación de Problemas II:

II.1 [Mismo signo -multiplicando-] (léalo después de haber visto en clase el condicional simple)

Solución → https://decsai.ugr.es/jccubero/FP/II_MismoSignoMultiplicando.cpp

Actividad: Resolución de problemas.

Se proporciona enlaces a videos que explican el esquema de la solución de algunos ejercicios. Consulte dichos videos **DESPUÉS** de haber intentado resolver el ejercicio por sus propios medios.

- *Obligatorios:*

  Ejercicios de la Relación de Problemas I:

  I.27 [Pasar de carácter a entero]

  Video → https://decsai.ugr.es/jccubero/FP/I_Caracter_to_Entero.mp4

  I.29 [Expresiones lógicas]

  Video → https://decsai.ugr.es/jccubero/FP/I_ExpresionesLogicas.mp4

  I.30 [Elección tipo de dato]

  Video → https://decsai.ugr.es/jccubero/FP/I_EleccionTipoDato.mp4

  Ejercicios de la Relación de Problemas II:

  II.2 [Se dividen]

  Video → https://decsai.ugr.es/jccubero/FP/II_SeDividen.mp4

  II.4 [Convertir mayúscula en minúscula (con un condicional)]

  II.5 [Tarifa aérea]

  Video → https://decsai.ugr.es/jccubero/FP/II_TarifaAerea.mp4

- *Opcionales:*

  II.7 [Operadores lógicos]

  Video → https://decsai.ugr.es/jccubero/FP/II_OperadoresLogicos.mp4

- *Complementarios:*

  Los siguientes ejercicios no tiene que entregarlos ni se defenderán. Si está empezando a programar le podrán ayudar a afianzar conceptos básicos.

  Ejercicios de la Relación de Problemas I:

  I.24 [Descuento tarifa aérea mezclando tipos]

  Solución → `https://decsai.ugr.es/jccubero/FP/I_TarifaAereaMezclandoTipos.cpp`

  I.25 [Lectura de un intervalo]

  Solución → `https://decsai.ugr.es/jccubero/FP/I_LecturaIntervalo.cpp`

  Ejercicios de la Relación de Problemas II:

  II.3 [Año bisiesto]

  Video → `https://decsai.ugr.es/jccubero/FP/II_Bisiesto.mp4`

  Solución → `https://decsai.ugr.es/jccubero/FP/II_Bisiesto.cpp`

  II.6 [Comparación de dos instantes (restando segundos)]

  Solución → `https://decsai.ugr.es/jccubero/FP/II_CompararInstantesRestando.cpp`



Fundamentos de Programación.

Relaciones de Problemas.

© CopyRight: Juan Carlos Cubero. Universidad de Granada.
Sugerencias: por favor, envíe un e-mail a
JC.Cubero@decsai.ugr.es

RELACIÓN DE PROBLEMAS DEL TEMA I. Introducción a C++

1. [Ley de Ohm] Cree un programa que pida un valor de intensidad y resistencia e imprima el voltaje correspondiente, según la *Ley de Ohm*:

   voltaje = intensidad * resistencia

   Finalidad: Ejemplo básico de asignación a una variable del resultado de una expresión. Dificultad Baja.

2. [Asignaciones secuenciales salario] Indique cuál sería el resultado de las siguientes operaciones:

```
int salario_base;
int salario_final;
int incremento;

salario_base = 1000;
incremento = 200;

salario_final = salario_base;
salario_final = salario_final + incremento;

salario_base = 3500;

cout << "\nSalario base:  " << salario_base;
cout << "\nSalario final: " << salario_final;
```

   Responda razonadamente a la siguiente pregunta: ¿El hecho de realizar la asignación `salario_final = salario_base;` hace que ambas variables estén ligadas durante todo el programa y que cualquier modificación posterior de `salario_base` afecte a `salario_final`?

   Finalidad: Ejemplo básico de asignación a una variable del resultado de una expresión. Dificultad Baja.

3. [Circunferencia] Cree un programa que nos pida la longitud del radio y calcule el área del círculo y la longitud de la circunferencia correspondientes. Finalmente, el programa mostrará en pantalla los resultados. Recuerde que:

   $$\text{área circulo} = \pi r^2 \qquad \text{longitud circunferencia} = 2\pi r$$

   Se proponen las siguientes alternativas.

   1. Pedir al usuario que introduzca el valor de π
   2. Usar el literal `3.1416` donde sea necesario.

3. Usar un dato llamado π, asignarle el literal `3.1416` y usarlo donde sea necesario.

   ¿Cuál elegiría y por qué?

   Finalidad: Uso de constantes. Dificultad Baja.

4. [Cálculo de π a partir del arco-seno] Si dividimos la longitud de cualquier circunferencia por el doble de su radio, se obtiene siempre el mismo número real, a saber $3.1415927\cdots$. Este número es bien conocido y se designa por la letra griega π. Es irracional y por tanto tiene infinitas cifras decimales. Hay diversos métodos para obtener este valor, algunos mejores que otros. En posteriores ejercicios veremos algunos de ellos. En la página web `https://en.wikipedia.org/wiki/Chronology_of_computation_of_pi` se puede consultar los últimos logros en la obtención de nuevas cifras decimales de π.

   Una forma de obtener π es a través de la función arco-seno, ya que $\frac{\pi}{6}$ es el ángulo en radianes de un seno de 0.5 (corresponde al seno de un ángulo de 30 grados):

   $$\frac{\pi}{6} = \text{arco-seno}(0.5)$$

   Construya un programa y declare una variable `PI`. Escriba una sentencia con la *ecuación* anterior. ¿Por qué se produce un error de compilación?

   Despeje el valor de π en la anterior ecuación y asígnele a la variable `PI` el valor correcto. Imprima en pantalla el resultado de evaluar la expresión. Debe mostrarse el valor `3.14159`.

   Recuerde que la función arco-seno está implementada en la función `asin` de la bilioteca `cmath` (como curiosidad, sepa que las calculadoras y librerías estándar de los compiladores implementan las funciones trigonométricas usando distintas variantes del algoritmo de Volder -buscar CORDIC en Internet-)

   Finalidad: Ejemplo básico de asignación a una variable del resultado de una expresión. Dificultad Baja.

5. [Variación porcentual] El *porcentaje de incremento* de una cantidad nos mide en qué proporción dicha cantidad ha experimentado un aumento. Por ejemplo, supongamos que el precio de un producto ha subido de 5 a 6 euros. En este caso, el porcentaje de incremento ha sido del 20 %. La fórmula matemática para obtenerlo es la siguiente:

   $$100 \times \frac{\text{valor final} - \text{valor inicial}}{\text{valor inicial}}$$

   En nuestro ejemplo, sería

   $$100 \times \frac{6-5}{5} = 20$$

   En el caso de que el valor final fuese menor que el valor inicial, se utiliza el término *porcentaje de decremento* . Se aplica la misma fórmula pero se cambia el signo para que

el resultado sea positivo. Por ejemplo, si un producto baja de 6 a 5 euros, el porcentaje de decremento sería $abs(100(6-5)/6) = 16.6667\,\%$. Observe que las cantidades involucradas son las mismas pero los porcentajes son distintos ya que influye quién es el valor inicial y final.

Para no hablar de aumentos o decrementos, en general, se define el concepto *variación porcentual* como:

$$VP = abs\left(100 \times \frac{\text{valor final} - \text{valor inicial}}{\text{valor inicial}}\right)$$

donde abs es la función valor absoluto.

Escriba un programa en C++ que lea el valor inicial y final de un producto (variables de tipo `double`) y calcule la variación porcentual del mismo.

▷ Ejemplo de entrada: 5 6
▷ Salida correcta: `variación porcentual: 20%`
▷ Ejemplo de entrada: 6 5
▷ Salida correcta: `variación porcentual: 16.6667%`

Finalidad: Ejemplo básico de asignación a una variable del resultado de una expresión. Dificultad Baja.

6. [Subir sueldo] Construya un programa para leer el valor de una variable `salario_base` de tipo `double`, la incremente un 2% e imprima el resultado en pantalla. Para realizar este cómputo, multiplique por `1.02` el valor original. Para resolver este ejercicio tiene varias alternativas:

   1. Directamente hacer el cómputo `1.02 * salario_base` dentro de la sentencia `cout`
   2. Introducir una variable `salario_final`, asignarle la expresión anterior y mostrar su contenido en la sentencia `cout`
   3. Modificar la variable original `salario_base` con el resultado de incrementarla un 2%.

   Indique qué alternativa elige y justifíquela.

   ▷ Ejemplo de entrada: 30 ▷ Salida correcta: `30.6`
   ▷ Ejemplo de entrada: 0 ▷ Salida correcta: 0

   Finalidad: Ejemplo básico de asignación a una variable del resultado de una expresión. Dificultad Baja.

7. [Dos subidas de sueldo] Recupere la solución del ejercicio I.6 [Subir sueldo] . Supondremos que la empresa aplica dos criterios distintos de subida salarial: uno es de antigüedad del 2% y otro de productividad del 3%. Se quiere aplicar ambas subidas salariales de dos formas distintas:

   1. *Conjunta*: Aplicando la suma de ambos incrementos sobre el salario inicial. Así pues, el incremento salarial final será del 5%.
   2. *Secuencial*: Aplicando el incremento del 3% sobre el resultado de haber aplicado antes el incremento del 2%.

   El programa debe mostrar los salarios resultantes, es decir: el resultante del incremento conjunto y el resultante del incremento secuencial.

   Debe usar índices de variación para gestionar las subidas salariales y debe definir las variables necesarias para que el programa sea fácil de interpretar y modificar en el futuro.

   ▷ Ejemplo de entrada: 30 ▷ Salida correcta: `31.5 31.518`

   ▷ Ejemplo de entrada: 0 ▷ Salida correcta: 0 0

   Finalidad: Trabajar con expresiones numéricas y con variables para no repetir cómputos. Dificultad Baja.

8. [Interés bancario] Un banco presenta la siguiente oferta. Si se deposita una cantidad de euros dada por la variable `capital` durante un año a plazo fijo, se dará un interés dado por la variable `interes`. Realice un programa que lea una cantidad `capital` y un interés `interes` desde teclado. A continuación, el programa debe calcular en una variable `total` el dinero que se tendrá al cabo de un año, aplicando la fórmula de abajo e imprimirá el resultado en pantalla.

$$\texttt{total} = \texttt{capital} + \texttt{capital} * \frac{\texttt{interes}}{100}$$

   Utilice el tipo de dato `double` para todas las variables. Supondremos que el usuario introduce el interés como un valor real entre 0 y 100, es decir, un interés del $5.4\,\%$ se introducirá como `5.4`. También supondremos que lo introduce correctamente, es decir, que sólo introducirá valores entre 0 y 100.

   Observe que para implementar la fórmula anterior, debemos usar el operador de división que en C++ es /, por lo que nos quedaría:

```
total = capital + capital * interes / 100;
```

   Es importante destacar que el compilador primero evaluará la expresión de la parte derecha de la anterior asignación (usando el valor que tuviese la variable `capital`) y a continuación ejecutará la asignación, escribiendo el valor resultante de la expresión dentro de la variable `total`.

   En la asignación que calcula la variable `total`, ¿se podría sustituir dicha variable por `capital`? es decir:

```
capital = capital + capital * interes / 100;
```

Analice las ventajas o inconvenientes de hacerlo así.

▷ Ejemplo de entrada: `300  5.4` ▷ Salida correcta: `316.2`

▷ Ejemplo de entrada: `300  0` ▷ Salida correcta: `300`

▷ Ejemplo de entrada: `0  5.4` ▷ Salida correcta: `0`

Finalidad: Resolver un problema real sencillo, usando varias sentencias. Dificultad Baja.

9. [Distancia Euclídea] Cree un programa que lea las coordenadas de dos puntos $P_1 = (x_1, y_1)$ y $P_2 = (x_2, y_2)$ y calcule la distancia euclídea entre ellos:

$$d(P_1, P_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Debe calcular el cuadrado sin usar ninguna función de la biblioteca `cmath`. Para la raíz cuadrada, sí puede usar la función `sqrt`.

▷ Ejemplo de entrada: `2.1  3.2  0.5  1.6` ▷ Salida correcta: `2.26274`

▷ Ejemplo de entrada: `2.1  3.2  2.1  3.2` ▷ Salida correcta: `0`

Finalidad: Trabajar con expresiones numéricas y con variables para no repetir cómputos. Dificultad Baja.

10. [Conversión de grados a radianes] Queremos transformar una cantidad g dada en grados a radianes. Para ello, basta aplicar la siguiente fórmula:

$$r = g\frac{\pi}{180}$$

Construya un programa que lea **dos** números reales que representarán dos grados. Debe calcular los radianes correspondientes en sendas variables e imprimir el resultado en pantalla.

▷ Ejemplo de entrada: `20  90`
▷ Salida correcta: `0.349066  1.5708`

▷ Ejemplo de entrada: `180  0`
▷ Salida correcta: `3.14159  0`

Para calcular π utilice la fórmula indicada en el ejercicio I.4 [Cálculo de π a partir del arco-seno] . Preste especial atención a no repetir cómputos que ya se hubiesen realizado anteriormente.

Finalidad: Ejemplo básico de no repetición de código. Dificultad Baja.

11. [Decimal redondeado] En `cmath` está definida la función `round` que permite redondear un real al entero más próximo. Por ejemplo:

`round(3.6)` devuelve `4`

`round(3.5)` devuelve `4`

`round(3.1)` devuelve 3

`round(3.49)` devuelve 3

Construya un programa para redondear un número real a cualquier cifra decimal. Por ejemplo:

El resultado de redondear `3.49` a la primera cifra decimal es `3.5`

El resultado de redondear `3.49` a la segunda cifra decimal es `3.49`

El resultado de redondear `3.496` a la segunda cifra decimal es `3.5`

El programa principal leerá el número real y la posición de la cifra decimal e imprimirá el resultado. Puede usar la misma función `round` y la función `pow`, ambas incluidas en `cmath`. La función `pow` eleva un número a otro; por ejemplo, para elevar `3.5` a 7, basta usar la expresión `pow(3.5, 7)`

▷ Ejemplo de entrada: `3.49 1` ▷ Salida correcta: `3.5`

▷ Ejemplo de entrada: `3.49 2` ▷ Salida correcta: `3.49`

▷ Ejemplo de entrada: `3.496 2` ▷ Salida correcta: `3.5`

Finalidad: Practicar la construcción de expresiones matemáticas. Dificultad Baja.

12. [Tarifa aérea según km] Una compañía aérea establece el precio del billete como sigue: en primer lugar se fija una tarifa base de 150 euros, la misma para todos los destinos. A continuación se suman 10 céntimos por cada kilómetro de distancia al destino.

    Cree un programa que lea el número de kilómetros al destino y calcule el precio final del billete.

    Utilice el tipo de dato `double` para todas las variables del programa. Este es un ejercicio docente; recuerde que, en un programa en producción, jamás usaremos tipos de datos reales (con representación en coma flotante como es el caso del tipo `double`) para representar una cantidad monetaria.

    ▷ Ejemplo de entrada: `120` ▷ Salida correcta: `162`

    ▷ Ejemplo de entrada: `200` ▷ Salida correcta: `170`

    ▷ Ejemplo de entrada: `201` ▷ Salida correcta: `170.1`

    ▷ Ejemplo de entrada: `452` ▷ Salida correcta: `195.2`

    Finalidad: Trabajar con expresiones numéricas. Dificultad Baja.

13. [Tarifa aérea con descuento] Una compañía aérea quiere aplicar una política de descuentos al precio final de un billete. Por un lado, si el número de puntos del cliente es alto se le aplicará un descuento del 4%. Por otro lado, si es un trayecto largo, se le aplicará un descuento del 2%. En este ejercicio no vamos a detectar si el cliente tiene muchos o pocos puntos o si el trayecto es largo o corto. Lo único que se le pide es que construya un programa que lea el precio del billete y aplique e imprima el resultado de aplicar los

dos posibles descuentos: el descuento por los puntos (sobre el precio de partida) y el descuento por el trayecto (sobre el precio de partida). Observe que los descuentos no se acumulan, sino que se aplican ambos sobre el mismo precio de partida.

Utilice el tipo de dato `double` para todas las variables del programa (en un programa real no usaríamos un tipo real para trabajar con cantidades monetarias por el error de redondeo).

▷ Ejemplo de entrada: `300` ▷ Salida correcta: `288  294`

▷ Ejemplo de entrada: `500` ▷ Salida correcta: `480  490`

▷ Ejemplo de entrada: `0` ▷ Salida correcta: `0  0`

Finalidad: Ejemplo básico de asignación a una variable del resultado de una expresión. Dificultad Baja.

14. [Segundos entre dos instantes] Calcule el número de segundos que hay entre dos instantes del mismo día.

    Cada instante se caracteriza por la hora (entre 0 y 23), minuto (entre 0 y 59) y segundo (entre 0 y 59).

    El programa leerá la hora, minuto y segundo del instante inicial y la hora, minuto y segundo del instante final (supondremos que los valores introducidos son correctos) y mostrará el número de segundos entre ambos instantes. Si el segundo instante es menor que el primero, se devolverá el número de segundos en negativo.

    ▷ Ejemplo de entrada: `9 12 9  10 34 55` ▷ Salida correcta: `4966`

    ▷ Ejemplo de entrada: `10 34 55  9 12 9` ▷ Salida correcta: `-4966`

    ▷ Ejemplo de entrada: `10 34 55  10 34 55` ▷ Salida correcta: `0`

    Finalidad: Trabajar con expresiones numéricas y algoritmos. Dificultad Baja.

15. [Uso de constantes] Re-escriba las soluciones de los ejercicios I.3 [Circunferencia] , I.4 [Cálculo de π a partir del arco-seno] , I.10 [Conversión de grados a radianes] , I.12 [Tarifa aérea según km] y I.13 [Tarifa aérea con descuento] usando constantes donde estime necesario.

    Finalidad: Uso de constantes. Dificultad Baja.

16. [Intercambiar variables] Queremos construir un programa que simule un juego inspirado en el de los *triles* (del que procede el nombre de *trilero*). Suponemos que hay dos participantes y cada uno tiene una caja etiquetada con su nombre. Dentro de cada caja hay una cantidad de dinero y el objetivo es intercambiar las cantidades que hay dentro. Por ahora, sólo se pide construir un programa que haga lo siguiente:

    - Debe leer desde teclado dos variables `caja_izda` y `caja_dcha`.
    - A continuación debe intercambiar sus valores y finalmente, mostrarlos en pantalla.

Observe que se desea intercambiar el contenido de las variables, de forma que `caja_izda` pasa a contener lo que tenía `caja_dcha` y viceversa. El siguiente código no es válido ya que simplemente engaña al usuario pero las cajas no se quedan modificadas:

```
cout << "La caja izquierda vale " << caja_dcha << "\n";
cout << "La caja derecha vale "   << caja_izda;
```

Estaríamos tentados a escribir el siguiente código:

```
caja_izda = caja_dcha;
caja_dcha = caja_izda;
```

pero no funciona correctamente ¿Por qué? Proponga una solución e impleméntela.

Finalidad: Entender cómo funciona la asignación entre variables. Dificultad Baja.

17. [Sistema métrico] Realice un programa que nos pida una longitud cualquiera dada en metros. El programa deberá calcular e imprimir en pantalla el equivalente de dicha longitud en pulgadas, pies, yardas y millas. Para el cálculo, utilice la siguiente tabla de conversión del sistema métrico:

1 pulgada= 25,4 milímetros
1 pie = 30,48 centímetros
1 yarda = 0,9144 metros
1 milla = 1609,344 metros
1 milla marina = 1852 metros

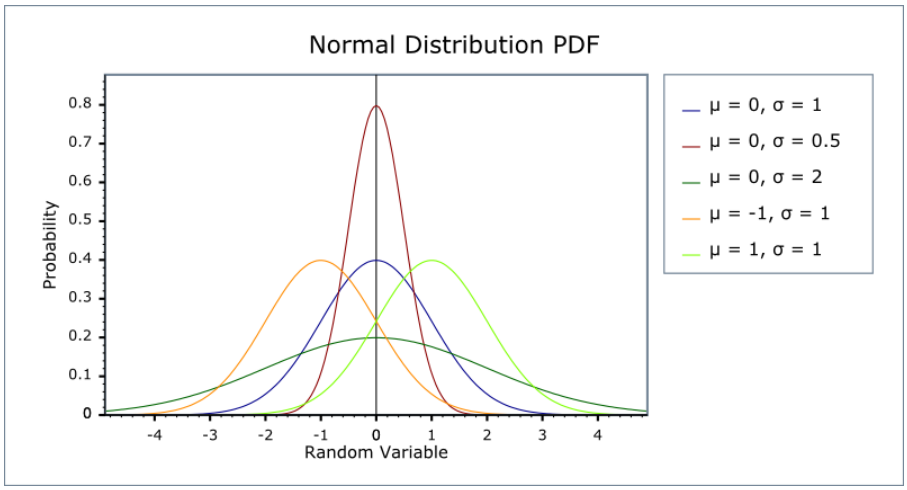
▷ Ejemplo de entrada: 1
▷ Salida correcta: `39.3701 3.28084 1.09361 0.00062`

Finalidad: Plantear la solución de un ejercicio básico como es el de una conversión. Dificultad Baja.

18. [Gaussiana] La función gaussiana es muy importante en Estadística. Es una función real de variable real que depende de dos parámetros μ y σ. El primero (μ) se conoce como *esperanza* o *media* y el segundo (σ) como *desviación típica* (*mean* y *standard deviation* en inglés). Su definición viene dada por la siguiente expresión:

$$\text{gaussiana}(x) = \frac{1}{\sigma\sqrt{2\pi}} \; e^{\left(-0.5\left(\frac{x-\mu}{\sigma}\right)^2\right)}$$

En la gráfica de abajo pueden verse algunos ejemplos de esta función con distintos parámetros.



Realice un programa que lea los coeficientes reales μ y σ de una función gaussiana. Dichos valores se conocen con el nombre de *esperanza* y *desviación* respectivamente. A continuación el programa leerá un valor de abscisa x y se imprimirá el valor que toma la función en x

Para realizar las operaciones indicadas , debe utilizar las siguientes funciones de `cmath`:

- Para elevar el número e a un valor cualquiera, use la función `exp`. Por ejemplo, para calcular e^8 debería usar la siguiente expresión:

  ```
  exp(8)
  ```

- Para calcular la raíz cuadrada, use `sqrt`.

- Para elevar un número a otro, utilice la función `pow` en la siguiente forma:

  ```
  pow(base, exponente)
  ```

  En nuestro caso, la base es $\frac{x-\mu}{\sigma}$ y el exponente 2.

  Una vez resuelto el ejercicio usando la función `pow`, resuélvalo de otra forma en la que no necesite usar dicha función.

Compruebe que los resultados son correctos, usando la calculadora disponible en:
https://www.medcalc.org/manual/normal_distribution_functions.php
Rellene las cajas de texto que aparecen en `PDFNormal` en el orden siguiente: en primer lugar, el valor de x, a continuación la esperanza (μ) y por último la desviación típica (σ). Por ejemplo, para ver el valor de la función gaussiana con esperanza 12 y desviación 5 en el punto 2.5, debe poner en la calculadora online lo siguiente y a continuación hacer click en el signo =:

```
PDFNormal(2.5, 12, 5) = 0.013123162955
```

Tenga en cuenta que en el programa de C++ los datos se le piden en otro orden: primero

la media y la desviación y luego el valor de la abscisa.

▷ Ejemplo de entrada: `12 5 2.5` ▷ Salida correcta: `0.01312316`

▷ Ejemplo de entrada: `0 1 0` ▷ Salida correcta: `0.39894228`

Finalidad: Trabajar con expresiones numéricas más complejas. Dificultad Media.

19. [Intercambiar tres variables] Se quiere generalizar el ejercicio I.16 que intercambiaba el valor de dos variables al caso de tres variables. Construya un programa que declare las variables `x`, `y`, `z`, lea su valor desde teclado e intercambien entre sí sus valores de forma que el valor de `x` pasa a `y`, el de `y` pasa a `z` y el valor de `z` pasa a `x` (se pueden declarar variables auxiliares aunque se pide que se use el menor número posible).

    ▷ Ejemplo de entrada: 7 4 5 ▷ Salida correcta: 5 7 4

    Finalidad: Mostrar la importancia en el orden de las asignaciones. Dificultad Media.

20. [Media y desviación] Escriba un algoritmo para calcular la media aritmética muestral y la desviación estándar (o típica) muestral de las alturas de tres personas (`n`=3). Estos valores serán reales (de tipo `double`). La fórmula general para un valor arbitrario de `n` es:

$$\overline{X} = \frac{1}{n}\sum_{i=1}^{n} x_i \ , \ S = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{X})^2}$$

    $\overline{X}$ representa la media aritmética y S la desviación típica muestral. Para resolver este problema es necesario usar la función `sqrt` (raíz cuadrada) que se encuentra en `cmath`.

    Estas medidas se utilizan mucho en Estadística para tener una idea de la distribución de datos. La media (mean en inglés) nos da una idea del valor central y la desviación típica (standard deviation) nos da una idea de la dispersión de éstos. Ejecutad el programa con varios valores y comprobad que el resultado es correcto utilizando una calculadora científica o cualquier calculadora online como por ejemplo la disponible en http://www.disfrutalasmatematicas.com/datos/desviacion-estandar-calculadora.html

    ▷ Ejemplo de entrada: `160 170 180`
    ▷ Salida correcta: `Media: 170 Desviación: 8.16497`
    ▷ Ejemplo de entrada: `170 170 170`
    ▷ Salida correcta: `Media: 170 Desviación: 0`

    Finalidad: Trabajar con expresiones numéricas y con variables para no repetir cómputos. Dificultad Baja.

21. [Aproximación del valor de π] Escriba un programa que muestre en pantalla el resultado de las siguientes expresiones numéricas que constituyen una aproximación al valor de π. La primera es del año 1800 antes de Cristo, la segunda (también de la misma era) es una aproximación introducida por los matemáticos mesopotámicos y la tercera es del

siglo II (introducida por Claudio Ptolomeo)

$$\pi \approx \frac{256}{81} \qquad \pi \approx 3 + \frac{1}{8} \qquad \pi \approx \frac{377}{120}$$

El resultado debe ser `3.16049`, `3.125` y `3.14167`

Finalidad: Mezclar tipos enteros y reales. Dificultad Baja.

22. [Gasolina] Escriba un programa que nos informe sobre el consumo de gasolina de un vehículo. Pedirá la distancia recorrida (en kms), los litros de gasolina consumidos y los litros que quedan en el depósito. El programa debe calcular e imprimir en pantalla el número de kilómetros realizados por cada litro consumido, el número de litros consumidos cada 100 kms y cuántos kilómetros de autonomía le restan con ese nivel de consumo. Utilice nombres de variables significativos.

    ▷ Ejemplo de entrada: `100 7 30`

    ▷ Salida correcta:

    ```
    Eficiencia del trayecto realizado: 14.2857 kms por litro
    Consumo del trayecto realizado:    7 litros cada 100 kms
    Autonomía restante:                428.571 kms
    ```

    Finalidad: Resolver un problema real sencillo, usando varias sentencias. Dificultad Baja.

23. [Índice de mayúscula] Realice un programa que lea una mayúscula desde teclado sobre una variable de tipo `char`. A continuación, el programa imprimirá el 0 si se ha introducido el carácter `A`, el 1 si era la `B`, el 2 si era la `C` y así sucesivamente. Supondremos que el usuario introduce siempre un carácter mayúscula.

    ▷ Ejemplo de entrada: `C` ▷ Salida correcta: 2

    Finalidad: Entender el tipo de dato `char`. *Dificultad Baja.*

24. [Descuento tarifa aérea mezclando tipos] Recupere la solución del ejercicio I.13 [Tarifa aérea con descuento] . En vez de fijar los dos descuentos del 4% y del 2% dentro del código, se quiere leer dichos valores con `cin`. Utilice el tipo de dato `int` para leer las dos variables correspondientes a los dos descuentos y el tipo `double` para el precio del billete.

    ▷ Ejemplo de entrada: `300.4 4 2` ▷ Salida correcta: `288.384 294.392`

    ▷ Ejemplo de entrada: `500 4 2` ▷ Salida correcta: `480 490`

    ▷ Ejemplo de entrada: `0 4 2` ▷ Salida correcta: `0 0`

    Finalidad: Mezclar tipos de datos numéricos en una expresión. Dificultad Baja.

25. [Lectura de un intervalo] Un intervalo es un espacio métrico comprendido entre dos valores o cotas, `a` y `b`, siendo `a` la cota inferior y `b` la cota superior. Cada extremo de un intervalo puede ser abierto o cerrado.

    En este problema, no se consideran intervalos con extremos infinitos como por ejemplo

$(-\infty, \infty)$.

Construya un programa que lea los datos de un intervalo. Para ello, el programa debe leer los datos en el siguiente orden:

1. Un carácter que represente el tipo de intervalo por la izquierda: el usuario deberá introducir el carácter `'('` si es abierto y `'['` si es cerrado.
2. Un número real con la cota izquierda.
3. El carácter `','` como separador de las dos cotas.
4. Un número real con la cota derecha.
5. Un carácter que represente el tipo de intervalo por la derecha: el usuario deberá introducir el carácter `')'` si es abierto y `']'` si es cerrado.

El programa mostrará en pantalla el mismo intervalo introducido.

▷ Ejemplo de entrada: `(  3.5 ,  5.1  ]` ▷ Salida correcta: `(3.5 , 5.1]`

Finalidad: Manejar entrada de datos de distinto tipo. Dificultad Baja.

26. [Pasar de mayúscula a minúscula] Construya un programa que lea un carácter (supondremos que el usuario introduce una mayúscula) en una variable de tipo `char` llamada `mayuscula`, la transforme a la minúscula correspondiente almacenando el resultado en una variable `minuscula` y lo imprima en pantalla. IMPORTANTE: Hágalo sin usar las funciones `toupper` ni `tolower` declaradas en `cctype`. Para ello, debe considerarse la relación que hay en C++ entre los tipos enteros y caracteres.

    ▷ Ejemplo de entrada: `D`

    ▷ Salida correcta: `d`

    Finalidad: Entender la equivalencia de C++ entre tipos enteros y de carácter. Dificultad Baja.

27. [Pasar de carácter a entero] Supongamos el siguiente código:

```
int entero;
char caracter;

cin >> caracter;
entero = caracter;
```

Supongamos que ejecutamos el código e introducimos el 7 desde teclado. El programa está leyendo una variable de tipo `char`. Por lo tanto, el símbolo 7 se interpreta como un carácter y es como si hiciésemos la siguiente asignación:

```
caracter = '7';
entero = caracter;
```

por lo que la variable `caracter` almacenará internamente el valor 55 (el orden en la tabla ASCII del carácter '7'). Lo mismo ocurre con la variable `entero`, que pasa a contener 55.

Sin embargo, queremos construir un programa para asignarle a la variable `entero` el número 7 asociado al dígito representado en la variable `caracter`, es decir, el 7 y no el 55. ¿Cómo se le ocurre hacerlo? El programa también imprimirá en pantalla el resultado.

Nota. La comilla simple para representar un literal de carácter es la que hay en el teclado del ordenador debajo de la interrogación `?`. Esta comilla hay que ponerla en el código pero no en la entrada del carácter desde teclado.

▷ Ejemplo de entrada: 7 (`cin` de un `char`) ▷ Salida correcta: 7 (`cout` de un `int`)

Finalidad: Entender la equivalencia de C++ entre tipos enteros y de carácter. Dificultad Baja.

28. [Escalado] Queremos transformar un entero `n` en el rango `{min, max}` y convertirlo en un entero `k` en el rango `{0, top}`, de forma que la posición de `n` con respecto a los extremos del rango `{min, max}` sea similar a la posición de `k` con respecto a los extremos de `{0, top}`. Por ejemplo, si partimos del rango `{9, 88}` y queremos transformar el valor 45 en un valor `k` en el rango `{0, 3}`, tenemos 4 posibilidades, a saber `0, 1, 2, 3`. De todas ellas, la más adecuada sería 1 ya que 45 está más cerca de 9 que de 88.

    Para ello, en primer lugar, debe transformar `n` en un valor *real* en el intervalo `[0,1]` y luego convertirlo a un entero en el rango `{0, top}`.

    Escriba un programa en C++ que lea los valores `min, max, top, n`, realice la conversión (escalado) pedida de `n` e imprima en pantalla el valor de `k`.

    Si lo desea, puede usar el esbozo del programa que puede encontrar en el siguiente enlace:
    https://decsai.ugr.es/jccubero/FP/I_EscaladoEsbozo.cpp

    ▷ Ejemplo de entrada: 9 88 3 45
    ▷ Salida correcta: 1
    ▷ Ejemplo de entrada: 9 88 3 71
    ▷ Salida correcta: 2
    ▷ Ejemplo de entrada: 9 88 3 9
    ▷ Salida correcta: 0
    ▷ Ejemplo de entrada: 9 88 3 88
    ▷ Salida correcta: 3

    Finalidad: Manejar expresiones con distintos tipos de datos numéricos. Dificultad Baja.

29. [Expresiones lógicas] La finalidad de este ejercicio es que empiece a construir expresiones lógicas no triviales. Para centrarnos en este objetivo y como excepción a lo que normalmente haremos, *puede usar directamente literales sin necesidad de definir datos*

de tipo `const`.

Tenga en cuenta que este es un ejercicio de la relación de problemas I por lo que no puede usar estructuras condicionales (se ven en el Tema II).

Se pide que cree un fichero de texto (con extensión `txt` o bien `cpp`) que contenga:

- Una expresión lógica que sea verdadera si una variable de tipo carácter llamada `letra` es una letra minúscula y falso en otro caso. Hágalo sin usar las funciones `islower`, `isupper` de la biblioteca `cctype`.
- Una expresión lógica que sea verdadera si una variable de tipo entero llamada `edad` es mayor o igual que 18 y menor o igual de 65 y falso en otro caso.
- Una expresión lógica que sea verdadera si una variable de tipo entero llamada `adivine` está entre 1 y 100 y falso en otro caso.
- Una expresión lógica que sea verdadera si un año es bisiesto y falso en otro caso. Los años bisiestos son aquellos que o bien son divisibles por 4 pero no por 100, o bien son divisibles por 400. Por ejemplo, son bisiestos: 1600,1996, 2000, 2004. No son bisiestos: 1700, 1800, 1900, 1998, 2002.
- Una expresión lógica que sea verdadera si un dato de velocidad es mayor o igual que 100 kilómetros por hora y falso en otro caso.
- Una expresión lógica que sea verdadera si un carácter es una vocal (sólo se consideran vocales las minúsculas sin acentos) y falso en otro caso.
- Una expresión lógica que sea verdadera si una recta está *inclinada*, es decir, que no sea ni horizontal ni vertical. Suponemos que una recta viene dada por tres coeficientes A, B, C $(Ax + By + C = 0)$. Es horizontal si $A = 0$ y es vertical si $B = 0$. Añada también una expresión lógica que sea verdadera si la recta está correctamente especificada, es decir, que no sean simultáneamente cero los coeficientes A y B.

Finalidad: Empezar a trabajar con expresiones lógicas, muy usadas en el tema 2. Dificultad Baja.

30. [Elección tipo de dato] Indique qué tipo de dato usaría para representar:

- Edad de una persona
- Producto interior bruto de un país. Consultad: http://es.wikipedia.org/wiki/Anexo:Pa%C3%ADses_por_PIB_(nominal)
- La cualidad de que un número entero sea primo o no.
- Estado civil (casado, soltero, separado, viudo)

- Sexo de una persona (hombre o mujer exclusivamente)

Finalidad: Saber elegir adecuadamente un tipo de dato, atendiendo a la información que se quiere representar. Dificultad Media.

31. [Precisión y desbordamiento] Indique si se produce un problema de precisión o de desbordamiento en los siguientes ejemplos y diga cuál sería el resultado final de las operaciones.

    Nota. Si se desea ver el contenido de una variable real con `cout`, es necesario que antes de hacerlo, se establezca el número de decimales que se quieren mostrar en pantalla. Para ello, basta ejecutar la sentencia `cout.precision(numero_digitos);` al inicio del programa. Hay que destacar que al trabajar con reales (representación en coma flotante como `double`, `float`, etc) siempre debemos asumir que el valor almacenado es sólo una representación aproximada.

    1.
    ```
    int chico, chico1, chico2;
    chico1 = 1234567;
    chico2 = 1234567;
    chico = chico1 * chico2;
    ```

    2.
    ```
    long grande;
    int chico1, chico2;
    chico1 = 1234567;
    chico2 = 1234567;
    grande = chico1 * chico2;
    ```

    3.
    ```
    double resultado, real1, real2;
    real1 = 123.1;
    real2 = 124.2;
    resultado = real1 * real2;
    ```

    4.
    ```
    double resultado, real1, real2;
    real1 = 123456789.1;
    real2 = 123456789.2;
    resultado = real1 * real2;
    ```

    5.
    ```
    double real, otro_real;
    real = 2e34;
    otro_real = real + 1;
    otro_real = otro_real - real;
    ```

    6.
    ```
    double real, otro_real;
    ```

```
real = 1e+300;
otro_real = 1e+200;
otro_real = otro_real * real;
```

7.
```
float chico;
double grande;
grande = 2e+150;
chico = grande;
```

Finalidad: Entender los problemas de desbordamiento y precisión. Dificultad Media.

Ejercicios adicionales (Introducción a C++)

32. [Cambio en monedas] Escribir un programa para gestionar el cambio automático de una caja de un supermercado. El programa debe leer el total a pagar y la cantidad pagada. Ambas cantidades se expresarán en céntimos y serán valores enteros. Supondremos que el usuario introduce correctamente los valores y por tanto la cantidad pagada es mayor o igual que el total a pagar. El programa debe calcular e imprimir en pantalla el número de monedas de 1 euro, 20 céntimos, 5 céntimos y 1 céntimo que se deben dar de cambio. El cambio debe ser tal que se utilice el menor número de monedas de cada clase.

    Finalidad: Resolver un problema real sencillo, usando varias sentencias. Dificultad Baja.

33. [Métricas atletismo] En atletismo se expresa la rapidez de un atleta en términos de ritmo (*minutos y segundos por kilómetro*) más que en unidades de velocidad (*kilómetros por hora*).

    Escriba dos programas para convertir entre estas dos medidas:

    1. El primero leerá el ritmo (minutos y segundos, por separado) y mostrará la velocidad (kilómetros por hora).
    2. El segundo leerá la velocidad (kilómetros por hora) y mostrará el ritmo (minutos y segundos).

    Finalidad: Trabajar con expresiones numéricas y con variables de diferentes tipos. Dificultad Baja.

34. [Reparto de ganancias] Las ganancias de un determinado producto se reparten entre el diseñador y los tres fabricantes del mismo. Diseñe un programa que pida la ganancia total de la empresa (los ingresos realizados con la venta del producto) y diga cuánto cobran cada uno de ellos, sabiendo que el diseñador cobra el doble que cada uno de los fabricantes. El dato de entrada será la ganancia total a repartir. Utilice el tipo `double` para todas las variables.
    Importante: No repita cálculos ya realizados.

    Finalidad: Entender la importancia de no repetir cómputos para evitar errores de programación. Dificultad Baja.

35. [Reparto de ganancias mezclando tipos] Realice el ejercicio del reparto de la ganancia de un producto, pero cambiando el tipo de dato de la ganancia total a `int` (el resto de variables siguen siendo `double`)

    Finalidad: Trabajar con expresiones numéricas que involucren distintos tipos de datos. Dificultad Baja.

36. Construya un programa que lea desde teclado tres variables correspondientes a un número de horas, minutos y segundos, respectivamente. A continuación, el programa debe calcular las horas, minutos y segundos dentro de su rango correspondiente. Por ejemplo, dadas 312 horas, 119 minutos y 1291 segundos, debería dar como resultado 13 días, 2 horas, 20 minutos y 31 segundos. El programa no calculará meses, años, etc. sino que se quedará en los días.

    Como consejo, utilice el operador `/` que cuando trabaja sobre datos enteros, obtiene la división entera. Para calcular el resto de la división entera, use el operador `%`.

    ▷ Ejemplo de entrada: `312 119 1291` ▷ Salida correcta: `13 2 20 31`

    Finalidad: Trabajar con expresiones numéricas y con variables para no repetir cómputos. *Dificultad Media.*

37. [Pinta dígitos] Escriba un programa que lea un valor entero e imprima en pantalla cada uno de sus dígitos separados por dos espacios en blanco. Supondremos que el usuario introduce siempre un entero de cinco dígitos, como por ejemplo `35197`. En este caso, la salida sería:

    ```
    3  5  1  9  7
    ```

    Finalidad: Ejemplo de asignación acumulada.

    Dificultad Media.

38. [Población] Los estudios poblacionales utilizan los conceptos de tasa de natalidad, mortalidad, etc. Al igual que un porcentaje representa una razón del total por cada cien (tanto por ciento), la tasa es una razón del total por cada mil (tanto por mil). Así pues una tasa de natalidad de 32, por ejemplo, significa que hay 32 nacimientos por cada 1000 habitantes.

    Escriba un programa que calcule la estimación de la población de un territorio después de tres años. Para ello, el programa debe leer la población inicial, la tasa de natalidad, la de mortalidad y la tasa de migración. Ésta última es la diferencia entre los que se van y los que vienen, por lo que puede ser o bien positiva o bien negativa.

    Suponga que todos los datos son enteros.

    Tenga en cuenta que una vez calculada la población que habrá el siguiente año, las tasas se vuelven a aplicar sobre la población así obtenida, y así sucesivamente, tantos años como estemos interesados.

    ▷ Ejemplo de entrada: `1375570814 32 12 7` ▷ Salida correcta: `1490027497`

    Finalidad: Ejemplo básico de asignación acumulada y uso de tipos numéricos distintos. *Dificultad Baja.*

39. [Desplaza entero dentro de un intervalo] Queremos construir una expresión numérica que desplace un entero un número de posiciones, pero lo mantenga dentro de un intervalo. Por ejemplo, si el intervalo fijado es `[65 , 90]`, el desplazamiento es de 3

unidades y el entero a desplazar es el 70, el resultado sería 73. Si el entero fuese el 89 y el desplazamiento 3, el resultado sería 92. Al no estar el 92 dentro del intervalo, se realiza un *ciclo* de forma que el 91 se transformaría en el 65 y el 92 en el 66.

Se pide construir un programa que lea dos enteros `minimo` y `maximo` que determinarán un intervalo `[minimo, maximo]`. Supondremos que el usuario introduce como `maximo` un valor mayor o igual que `minimo`. A continuación el programa leerá un valor entero `desplazamiento` (supondremos que el usuario introduce un valor entre 0 y `maximo - minimo`). Finalmente, el programa leerá un entero `a_desplazar` (supondremos que el usuario introduce un número entre `minimo` y `maximo`) le sumará el valor `desplazamiento` y lo convertirá en un entero dentro del intervalo `[minimo, maximo]` tal y como se ha descrito anteriormente.

▷ Ejemplo de entrada: `65 90 3 70` ▷ Salida correcta: `73`

▷ Ejemplo de entrada: `65 90 3 89` ▷ Salida correcta: `66`

▷ Ejemplo de entrada: `65 90 3 90` ▷ Salida correcta: `67`

Finalidad: Trabajar con los operadores enteros. Dificultad Media.

40. [Codificación de caracteres con algoritmo de rotación] Para intercambiar mensajes de forma privada, se utilizan distintos algoritmos que codifican/descodifican una cadena de caracteres. Uno de los más sencillos y que fue utilizado por Julio César durante la época del Imperio Romano es el de rotación.

    Consiste en seleccionar una `clave` (un número entero), y desplazar las letras del alfabeto tantas posiciones como indica la clave.

    Trabajaremos únicamente con las letras mayúsculas.

    Se considera una representación *circular* del alfabeto, de tal forma que el carácter que sigue a `'Z'` es `'A'`. Por ejemplo, si `clave=4`, entonces la `'A'` se reemplaza por la `'E'` y la `'Z'` por la `'D'`. Utilizando `clave=0` la secuencia cifrada es igual a la original.

    Construya un programa que lea un entero representando la clave y un carácter (supondremos que se introduce correctamente una letra mayúscula del alfabeto inglés) El programa codificará el carácter según la clave introducida y lo mostrará por pantalla.

    Para resolver este ejercicio sólo de pueden usar las herramientas de programación vistas en el primer tema. Se recomienda que revise la solución del ejercicio I.39 [Desplaza entero dentro de un intervalo] de esta Relación de Problemas.

    ▷ Ejemplo de entrada: `4 A` ▷ Salida correcta: `E`

    ▷ Ejemplo de entrada: `4 Y` ▷ Salida correcta: `C`

    Finalidad: Expresiones con caracteres y enteros. Dificultad Media.

41. [Coordenadas geográficas] En ejercicios posteriores, trabajaremos con un conjunto de coordenadas geográficas y necesitaremos realizar ciertas operaciones trigonométricas. En este ejercicio, vamos a implementar una parte de dichas operaciones.

    Se pide construir un programa que lea cuatro variables reales

grados_lat$_1$, grados_lon$_1$, grados_lat$_2$, grados_lon$_2$ representando las coordenadas (en grados) de longitud y latitud de dos puntos en el plano y calcule el valor de la variable a aplicando la siguiente fórmula:

$$a = \sin^2\left(\frac{1}{2}(\text{lat}_2 - \text{lat}_1)\right) + \cos(\text{lat}_1)\ \cos(\text{lat}_2)\ \sin^2\left(\frac{1}{2}(\text{lon}_2 - \text{lon}_1)\right)$$

Los valores de lat$_i$ y lon$_i$ son los valores correspondientes a grados_lat$_i$ grados_lon$_i$, pero expresados en radianes, por lo que tendrá que utilizar la conversión vista en el ejercicio I.10 [Conversión de grados a radianes]

Tenga en cuenta que si escribe la expresión `1/2` el resultado es cero ya que, al ser los operandos de tipo entero, el operador `/` es la división entera.

▷ Ejemplo de entrada: `37.18817  -3.60667  41.89193  12.51133`
▷ Salida correcta: `0.0133395`

Los datos anteriores corresponden a la latitud y longitud de Granada y Roma respectivamente.

Finalidad: Trabajar con expresiones numéricas y con variables para no repetir cómputos. Dificultad Baja.

42. [Coordenadas geográficas (distancia)] Recupere la solución del ejercicio I.41 [Coordenadas geográficas] de la Relación de Problemas I.

Se quiere calcular la distancia entre dos puntos de la Tierra. Cada punto vendrá determinado por sus coordenadas geográficas, dadas por tres datos reales: su longitud, latitud y altura. Lo vamos a hacer de dos formas:

1. Sin tener en cuenta la altura. La distancia *sobre plano* entre dos puntos se calcula como la longitud del segmento d_p de la figura y se puede calcular con la llamada *fórmula del Haversine*:

   a) Se calcula $\Delta_{\text{lon}} = \text{lon}_2 - \text{lon}_1$ y $\Delta_{\text{lat}} = \text{lat}_2 - \text{lat}_1$

   b) Se calcula $a = \sin^2\left(\frac{1}{2}\,\Delta_{\text{lat}}\right) + \cos(\text{lat}_1)\ \cos(\text{lat}_2)\ \sin^2\left(\frac{1}{2}\,\Delta_{\text{lon}}\right)$

   c) Se calcula $c = 2\ \arcsin(\text{mín}(1.0, \sqrt{a}))$ donde $\text{mín}(1, \sqrt{a}))$ representa el mínimo entre 1 y $\sqrt{a}$.

   d) Finalmente, la distancia será $d_p = R\,c$, donde $R = 6372797.560856$ metros es la longitud media del radio terrestre.

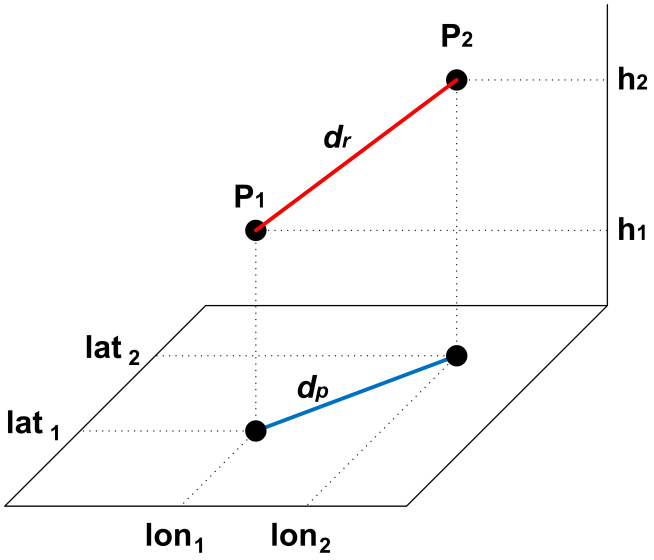
   Tenga en cuenta lo siguiente:

   a) Los datos de latitud y longitud en las fórmulas, vienen expresados en radianes.

   b) La función arco-seno ($\arcsin$) viene ya implementada en la biblioteca `cmath` con el nombre `asin`

c) $\mathbf{mín}$ representa el mínimo de dos valores. Para implementarlo utilice la función `min` que viene definida en la biblioteca `algorithm`.

2. Teniendo en cuenta la altura se obtiene la distancia *real* entre los dos puntos. Dicha distancia es la longitud del segmento d_r en la figura. Para calcular dicho valor, basta observar que se puede formar un triángulo rectángulo a partir de los segmentos d_p, d_r y la diferencia de las alturas (representadas por h1 y h2 en el gráfico) y aplicar el teorema de Pitágoras. La longitud del segmento d_p es la distancia euclídea entre los extremos del segmento.



Construya un programa que lea las coordenadas del primer punto (latitud, longitud y altura), las del segundo e imprima en pantalla la distancia sobre plano y la distancia real. Supondremos que las coordenadas de los puntos se introducen en grados, por lo que habrá que pasarlas a radianes para poder aplicar las fórmulas anteriores, tal y como hicimos en el ejercicio I.10 [Conversión de grados a radianes] de esta Relación de Problemas.

▷ Ejemplo de entrada: `37.18817 -3.60667 738 41.89193 12.51133 52`
▷ Salida correcta: `1475367.2005551  1475367.36003946`

Los datos anteriores corresponden a la latitud, longitud y altura de Granada y Roma respectivamente.

Para poder ver los reales con varios dígitos de precisión, incluya la sentencia `cout.precision(15);` al inicio del programa, por ejemplo, después de declarar las variables.

Finalidad: Trabajar con expresiones numéricas. Dificultad Baja.

43. Razone sobre la falsedad o no de las siguientes afirmaciones:

1. `'c'` es una expresión de caracteres.

2. `4 < 3` es una expresión numérica.
3. `(4 + 3) < 5` es una expresión numérica.
4. `cout << a;` da como salida la escritura en pantalla de una `a`.
5. ¿Qué realiza `cin >> cte`, siendo `cte` una constante entera?

Finalidad: Distinguir entre expresiones de distinto tipo de dato. Dificultad Baja.

44. El precio final de un automóvil para un comprador es la suma total del costo del vehículo, del porcentaje de ganancia de dicho vendedor y del I.V.A. Diseñe un algoritmo para obtener el precio final de un automóvil sabiendo que el porcentaje de ganancia de este vendedor es del 20 % y el I.V.A. aplicable es del 16 %.
Dificultad Baja.

45. Cree un programa que lea un valor de temperatura expresada en grados Celsius y la transforme en grados Fahrenheit. Para ello, debe considerar la fórmula siguiente:

```
Grados Fahrenheit = (Grados Celsius * 180 / 100) + 32
```

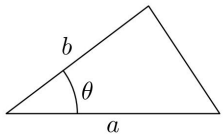
Buscad en Internet el por qué de dicha fórmula.
Dificultad Baja.

46. Dos locomotoras parten de puntos distintos avanzando en dirección contraria sobre la misma vía. Se pide redactar un programa para conocer las distancias que habrán recorrido ambas locomotoras antes de que choquen teniendo en cuenta que la primera locomotora viaja a una velocidad constante V_1, que la segunda viaja a una velocidad constante V_2, la fórmula que relaciona velocidad, espacio y tiempo ($s = v\ t$) y que el momento en que se producirá el choque viene dado por la fórmula

$$t = \frac{D}{V_1 + V_2}$$

dónde D es la distancia que separa los puntos iniciales de partida. Los datos de entrada al programa serán D, V_1 y V_2.
Dificultad Baja.

47. El área A de un triángulo se puede calcular a partir del valor de dos de sus lados, a y b, y del ángulo θ que éstos forman entre sí con la fórmula $A = \frac{1}{2}ab\ sin(\theta)$. Construya un programa que pida al usuario el valor de los dos lados (en centímetros), el ángulo que éstos forman (en grados), y muestre el valor del área.



Tened en cuenta que el argumento de la función `sin` va en radianes por lo que habrá que transformar los grados del ángulo en radianes (recordad que 360 grados son 2Π radianes).

Dificultad Baja.

48. Los compiladores utilizan siempre el mismo número de bits para representar un tipo de dato entero (este número puede variar de un compilador a otro). Por ejemplo, 32 bits para un `int`. Pero, realmente, no se necesitan 32 bits para representar el 6, por ejemplo, ya que bastarían 3 bits:

$$6 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0 \equiv 110$$

Se pide crear un programa que lea un entero n, y calcule el mínimo número de dígitos que se necesitan para su representación. Para simplificar los cómputos, suponed que sólo queremos representar valores enteros positivos (incluido el cero). Consejo: se necesitará usar el logaritmo en base 2 y obtener la parte entera de un real (se obtiene tras el truncamiento que se produce al asignar un real a un entero)

Dificultad Media.

RELACIÓN DE PROBLEMAS DEL TEMA II. Estructuras de Control

Problemas sobre condicionales

1. [Mismo signo -multiplicando-] Construya un programa que lea dos datos `a` y `b` de tipo `int` y nos diga si tienen el mismo signo. Se considera que el cero no tiene signo por lo que cualquier número (incluido el cero) tiene un signo distinto del cero.

   Para ello, basta comprobar si la multiplicación de `a` y `b` es positiva o no.

   ▷ Ejemplo de entrada: `0 0` ▷ Salida correcta: `No tienen el mismo signo`
   ▷ Ejemplo de entrada: `0 3` ▷ Salida correcta: `No tienen el mismo signo`
   ▷ Ejemplo de entrada: `3 0` ▷ Salida correcta: `No tienen el mismo signo`
   ▷ Ejemplo de entrada: `3 4` ▷ Salida correcta: `Tienen el mismo signo`
   ▷ Ejemplo de entrada: `-3 -4` ▷ Salida correcta: `Tienen el mismo signo`
   ▷ Ejemplo de entrada: `3 -4` ▷ Salida correcta: `No tienen el mismo signo`
   ▷ Ejemplo de entrada: `-3 4` ▷ Salida correcta: `No tienen el mismo signo`

   Finalidad: Trabajar con estructuras condicionales dobles. Dificultad Baja.

2. [Se dividen] Realice un programa que lea dos valores enteros desde teclado y diga si cualquiera de ellos divide o no (de forma entera) al otro. En este problema no hace falta decir quién divide a quién. Supondremos que los valores leídos desde teclado son ambos distintos de cero.

   ▷ Ejemplo de entrada: `3 4` ▷ Salida correcta: `Ninguno divide al otro`
   ▷ Ejemplo de entrada: `2 4` ▷ Salida correcta: `Uno de ellos divide al otro`
   ▷ Ejemplo de entrada: `4 2` ▷ Salida correcta: `Uno de ellos divide al otro`

   Finalidad: Plantear una estructura condicional doble con una expresión lógica compuesta. Dificultad Baja.

3. [Año bisiesto] Recuerde la definición de año bisiesto vista en el ejercicio I.29 [Expresiones lógicas] de la Relación de Problemas I. Construya un programa para que lea un año y nos diga si es o no bisiesto con un mensaje en pantalla.

   ▷ Ejemplo de entrada: `2017` ▷ Salida correcta: `No es bisiesto`
   ▷ Ejemplo de entrada: `2000` ▷ Salida correcta: `Es bisiesto`

   Finalidad: Plantear una estructura condicional simple con salida de resultados. Dificultad Baja.

4. [Convertir mayúscula en minúscula (con un condicional)] Se quiere leer un carácter

`letra_original` desde teclado, y comprobar con una estructura condicional si es una letra mayúscula. En dicho caso, hay que calcular la minúscula correspondiente almacenando el resultado en una variable llamada `letra_convertida`. En el caso de que no sea una mayúscula, le asignaremos a `letra_convertida` el valor que tenga `letra_original`. Finalmente, imprimiremos en pantalla el valor de `letra_convertida`. No pueden usarse las funciones `tolower` ni `toupper` de la biblioteca `cctype`.

▷ Ejemplo de entrada: `D` ▷ Salida correcta: `d`

▷ Ejemplo de entrada: `d` ▷ Salida correcta: `d`

▷ Ejemplo de entrada: `!` ▷ Salida correcta: `!`

Finalidad: Plantear una estructura condicional doble con una expresión lógica compuesta. Dificultad Baja.

5. [Tarifa aérea] Retome la solución del ejercicio I.12 [Tarifa aérea según km] de la Relación de Problemas I disponible en:

   https://decsai.ugr.es/jccubero/FP/I_TarifaAerea.cpp

   La forma de calcular la tarifa final del billete cambia ahora de la forma siguiente: la tarifa base sigue siendo de 150 euros, la misma para todos los destinos. Ahora bien, si el destino está a menos de 300 kilómetros, el precio final es la tarifa base. Para destinos a más de 300 Km, se suman 10 céntimos por cada kilómetro de distancia al destino (a partir del Km 301).

   Cree un programa para que lea el número de kilómetros al destino y calcule el precio final del billete.

   ▷ Ejemplo de entrada: `120` ▷ Salida correcta: `150`

   ▷ Ejemplo de entrada: `300` ▷ Salida correcta: `150`

   ▷ Ejemplo de entrada: `301` ▷ Salida correcta: `150.1`

   ▷ Ejemplo de entrada: `452` ▷ Salida correcta: `165.2`

   Finalidad: Plantear una estructura condicional simple. Actualizar una variable según una condición. Dificultad Baja.

6. [Comparación de dos instantes (restando segundos)] Construya un programa que lea dos instantes de tiempo (según se define en el ejercicio I.14 [Segundos entre dos instantes] de la Relación de Problemas I) y nos diga si el primero es anterior al segundo. Para ello, calcule los segundos que hay de diferencia entre ambos instantes, tal y como se hizo en el ejercicio I.14 [Segundos entre dos instantes] de la Relación de Problemas I.

   ▷ Ejemplo de entrada: `9 12 9  10 34 55`
   ▷ Salida correcta: `El primero es anterior`

   ▷ Ejemplo de entrada: `10 34 55  9 12 9`
   ▷ Salida correcta: `El primero no es anterior`

   ▷ Ejemplo de entrada: `10 34 55  10 34 55`

▷ Salida correcta: `El primero no es anterior`

Finalidad: Planteamiento de una estructura condicional simple. Dificultad Baja.

7. [Operadores lógicos] En este ejercicio no hace falta construir ningún programa. Debe crear un fichero de texto (con extensión `cpp` o `txt`) explicando qué problemas observa en los siguientes condicionales:

1.
```
char tipo_radar;
cin >> tipo_radar;

if (tipo_radar == 'F' && tipo_radar == 'f')
   ......
```

2.
```
double velocidad;
cin >> velocidad;

if (velocidad > 100 && velocidad < 70)
   cout << "\nVelocidad fuera del rango";
```

3.
```
double velocidad;
cin >> velocidad;

if (velocidad > 100 || velocidad > 110)
   cout << "Velocidad excesiva";
```

Finalidad: Entender cómo funcionan los operadores lógicos. Dificultad Baja.

8. [Tabulación] . Los siguientes trozos de código no están correctamente tabulados según las normas indicadas en clase. Re-escríbalos e intente simplificar las estructuras obtenidas.

a)
```
if (edad > 18)
es_mayor_edad = true;
else es_mayor_edad = false;
```

b)
```
if (edad > 18)
if (ingresos <= MAX_INGRESOS)
suscept_ayuda = true;
else suscept_ayuda = false;
else if (ingresos > MIN_INGRESOS)
suscept_ayuda = true;
else
suscept_ayuda = false;
```

Finalidad: Hacer énfasis en la importancia de la correcta tabulación del código. Dificultad Baja.

9. [Cambia capitalización] Queremos modificar el ejercicio II.4 [Convertir mayúscula en minúscula (con un condicional)] para leer un carácter `letra_original` desde teclado y hacer lo siguiente:

   - Si es una letra mayúscula, almacenaremos en la variable `letra_convertida` la correspondiente letra minúscula.
   - Si es una letra minúscula, almacenaremos en la variable `letra_convertida` la correspondiente letra mayúscula.
   - Si es un carácter no alfabético, almacenaremos el mismo carácter en la variable `letra_convertida`

   El programa debe imprimir en pantalla el valor de `letra_convertida` e indicar si la letra introducida era una minúscula, mayúscula o no era una carácter alfabético. Resuelva este ejercicio realizando los cómputos en un bloque de código y la salida de resultados en pantalla en otro bloque de código posterior. No pueden usarse las funciones `islower, tolower` `isupper, toupper` de la biblioteca `cctype`.

   ▷ Ejemplo de entrada: `D`
   ▷ Salida correcta: `El carácter D es una mayúscula. Se convierte en d`
   ▷ Ejemplo de entrada: `d`
   ▷ Salida correcta: `El carácter d es una minúscula. Se convierte en D`
   ▷ Ejemplo de entrada: `!`
   ▷ Salida correcta: `El carácter ! no es ni mayúscula ni minúscula. Se convierte en !`

   Finalidad: Trabajar con estructuras condicionales. Dificultad Baja.

10. [Tarifa aérea con descuentos] Recupere la solución del ejercicio II.5 [Tarifa aérea] de esta Relación de Problemas. Una vez se ha obtenido el precio del billete (con el incremento de 10 céntimos por km recorrido, en su caso), se quieren aplicar varios descuentos **acumulativos**.

    Un primer descuento vendrá determinado por el número de kilómetros del trayecto. El segundo descuento dependerá del número de puntos de la tarjeta de fidelización del cliente que ha comprado el billete. En concreto:

    1. Si el número de puntos es mayor de 100, se aplica un descuento del 3 % y si es mayor de 200, se aplica un descuento del 4 %.
    2. Si el trayecto es mayor de 700 km, se aplica un descuento del 2 %

    Los dos descuentos anteriores (por longitud del trayecto y el correspondiente al número

de puntos), son independientes y acumulables. En cualquier caso, ambos se aplican sobre el precio del billete (una vez le ha aumentado el recargo por el número de kilómetros), es decir, un descuento no se aplica sobre el resultado de haber aplicado previamente el otro descuento. Por lo tanto, un cliente podría beneficiarse de un descuento del 2%, del 3%, del 4%, del 5%, del 6%, o ninguno.

Construya un programa que lea el número de kilómetros del trayecto y el número de puntos del cliente e imprima en pantalla el precio final del billete con los descuentos aplicados, en su caso.

- ▷ Ejemplo de entrada: `200 90` ▷ Salida correcta: `150`
- ▷ Ejemplo de entrada: `200 120` ▷ Salida correcta: `145.5`
- ▷ Ejemplo de entrada: `200 250` ▷ Salida correcta: `144`
- ▷ Ejemplo de entrada: `650 90` ▷ Salida correcta: `185`
- ▷ Ejemplo de entrada: `650 120` ▷ Salida correcta: `179.45`
- ▷ Ejemplo de entrada: `800 90` ▷ Salida correcta: `196`
- ▷ Ejemplo de entrada: `800 120` ▷ Salida correcta: `190`
- ▷ Ejemplo de entrada: `800 250` ▷ Salida correcta: `188`

Finalidad: Planteamiento de una estructura condicional secuencial. Dificultad Baja.

11. [Velocidad imputada] La velocidad de los vehículos en las vías de circulación es captada por unos aparatos electrónicos conocidos como radares o *cinemómetros*. Éstos pueden ser fijos o móviles y presentan un margen de error que hay que aplicar a la velocidad captada por el radar. Dichos márgenes vienen especificados en la orden ITC/3123/2010, de 26 de noviembre
https://www.boe.es/diario_boe/txt.php?id=BOE-A-2010-18556

    1. En el caso de un radar fijo:

        a) Si la velocidad captada por el radar es menor o igual que 100 km/h, el margen de error es de $\pm$ 5km/h.

        b) En caso contrario, el margen de error es de un 5%

    2. En el caso de un radar móvil:

        a) Si la velocidad captada por el radar es menor o igual que 100 km/h, el margen de error es de 7km/h.

        b) En caso contrario, el margen de error es de un 7%

    Los márgenes de error se aplican sobre la velocidad captada y da como resultado la velocidad imputada. La aplicación se realiza en el sentido más ventajoso para el contribuyente, es decir, disminuyendo la velocidad captada. Por ejemplo, si la velocidad captada es de 95 km/h, la velocidad imputada sería de `95 - 5 = 90` km/h en el caso de un

radar fijo y de `95 - 7 = 88` km/h en el caso de un radar móvil.

Si la velocidad captada es, por ejemplo, de `104` km/h, la velocidad imputada sería de `104 - 5% de 104 = 104 - 5.2 = 98.8` km/h en el caso de un radar fijo y de `104 - 7% de 104 = 104 - 7.28 = 96.72` km/h en el caso de un radar móvil.

Construya un programa que lea desde teclado un carácter que indique el tipo de radar (`'F'` para fijo y cualquier otra letra para móvil), la velocidad captada, e imprima la velocidad imputada.

▷ Ejemplo de entrada: `F 95` ▷ Salida correcta: `90`
▷ Ejemplo de entrada: `M 95` ▷ Salida correcta: `88`
▷ Ejemplo de entrada: `F 104` ▷ Salida correcta: `98.8`
▷ Ejemplo de entrada: `M 104` ▷ Salida correcta: `96.72`

Nota:

En el ejercicio II.45 [Multa autovía] de esta Relación de Problemas, se usa la velocidad del vehículo para establecer la sanción correspondiente en una autovía (dicho ejercicio no tiene que resolverlo).

Finalidad: Plantear una estructura condicional anidada y actualizar variables en función de ciertas condiciones. Dificultad Media.

12. [Mismo signo con condicionales] Continuamos trabajando sobre la base del ejercicio II.1 [Mismo signo -multiplicando-] de esta Relación de Problemas.

    Resuelva el mismo problema de ver si dos enteros tienen el mismo signo sin utilizar la operación de multiplicación. Implemente las siguientes posibilidades:

    1. Enumere todas las posibles situaciones, es decir, los dos positivos, los dos negativos, el primero positivo y el segundo negativo o el primero negativo y el segundo positivo. Use varias estructuras condicionales consecutivas.

       Si resolvemos de esta forma el problema, el programa será muy propenso a errores. ¿Por qué?

    2. Utilice una estructura condicional anidada para no tener que enumerar todos los casos posibles.

    3. Construya una única expresión lógica y asigne directamente el resultado a una variable lógica.

    ▷ Ejemplo de entrada: `0 0` ▷ Salida correcta: `No tienen el mismo signo`
    ▷ Ejemplo de entrada: `0 3` ▷ Salida correcta: `No tienen el mismo signo`
    ▷ Ejemplo de entrada: `3 0` ▷ Salida correcta: `No tienen el mismo signo`
    ▷ Ejemplo de entrada: `3 4` ▷ Salida correcta: `Tienen el mismo signo`
    ▷ Ejemplo de entrada: `-3 -4` ▷ Salida correcta: `Tienen el mismo signo`

▷ Ejemplo de entrada: `3  -4` ▷ Salida correcta: `No tienen el mismo signo`
▷ Ejemplo de entrada: `-3  4` ▷ Salida correcta: `No tienen el mismo signo`

Finalidad: Trabajar con estructuras condicionales anidadas y expresiones lógicas compuestas. Dificultad Media.

13. [Comparación de dos instantes] Realice lo mismo que se le pide en el ejercicio II.6 [Comparación de dos instantes (restando segundos)] pero resolviéndolo de otra forma. En vez de calcular los segundos totales que hay en cada instante, tiene que usar los valores por separado de la hora, minuto y segundo del instante inicial y los tiene que comparar (usando expresiones lógicas) con los correspondientes del instante final.
    ▷ Ejemplo de entrada: `9 12 9  10 34 55`
    ▷ Salida correcta: `El primero es anterior`
    ▷ Ejemplo de entrada: `10 34 55  9 12 9`
    ▷ Salida correcta: `El primero no es anterior`
    ▷ Ejemplo de entrada: `10 34 55  10 34 55`
    ▷ Salida correcta: `El primero no es anterior`

    Finalidad: Planteamiento de una estructura condicional anidada. Dificultad Media.

14. [Intervalo] Recupere la solución del ejercicio I.25 [Lectura de un intervalo] de la Relación de Problema I. Si lo desea, puede usar el código disponible en el siguiente enlace:
    `https://decsai.ugr.es/jccubero/FP/I_LecturaIntervalo.cpp`
    Después de haber leído los datos que definen el intervalo, el programa debe leer un valor real y determinar si está o no dentro del intervalo.
    ▷ Ejemplo de entrada: `(3.5 , 5.1]  4.8`
    ▷ Salida correcta: `El valor 4.8 está dentro del intervalo (3.5, 5.1]`
    ▷ Ejemplo de entrada: `(3.5 , 5.1]  8.3`
    ▷ Salida correcta: `El valor 8.3 no está dentro del intervalo (3.5, 5.1]`
    ▷ Ejemplo de entrada: `(3.5 , 5.1]  3.5`
    ▷ Salida correcta: `El valor 3.5 no está dentro del intervalo (3.5, 5.1]`
    ▷ Ejemplo de entrada: `(3.5 , 5.1]  5.1`
    ▷ Salida correcta: `El valor 5.1 está dentro del intervalo (3.5, 5.1]`

    Finalidad: Plantear una condición compleja. Dificultad Media.

15. [Coronavirus] Por todos es conocido el gran daño humano y económico que ha producido el coronavirus `SARS-CoV-2` al provocar la enfermedad denominada `COVID-19`. En Argentina, la web gubernamental `https://coronavirus.argentina.gob.ar/` (actualmente no está disponible) diseñó un test aproximado para su identificación. El código fuente estaba escrito en JavaScript y parte de él se muestra en la imagen del siguiente enlace (como puede apreciar, Java es un lenguaje con una sintaxis similar a C++)

https://decsai.ugr.es/jccubero/FP/II_coranovirus.png

La lógica implementada en este código es correcta y por tanto identifica adecuadamente la enfermedad del COVID-19. Sin embargo, este código tiene varios problemas (a los pocos días de publicarlo arreglaron parte de ellos) en el sentido de que incumple algunas normas que un buen programador ha de seguir. Uno de los problemas más sencillos de detectar es la aparición duplicada de las comprobaciones sobre `respiratoryDisease`. En cualquier caso, para simplificar el problema nos vamos a fijar únicamente en cuatro síntomas, a saber, `bodyTemperature`, `difficultyBreathing`, `diabetes` y `cancer`, de forma que el código anterior se simplificaría en el siguiente:

```
if((bodyTemperature >= 38 && difficultyBreathing) ||
   (bodyTemperature >= 38 && difficultyBreathing && diabetes) ||
   (bodyTemperature >= 38 && difficultyBreathing && cancer) ||
   (bodyTemperature >= 38 && diabetes) ||
   (bodyTemperature >= 38 && cancer))
   cout << "Consulte autoridades locales";
else
   if (bodyTemperature >= 38)
      cout << "Cuarentena en su casa";
   else
      if (bodyTemperature < 38)
         cout << "Test negativo";
      else
         cout << "Test negativo";
```

Identifique los problemas que pueda haber y proponga una solución modificando el código del programa que puede encontrar en el siguiente enlace:

https://decsai.ugr.es/jccubero/FP/II_CoronavirusEsbozo.cpp

▷ Ejemplo de entrada: `37  S S S`
▷ Salida correcta: `Test negativo`

▷ Ejemplo de entrada: `39  S N N`
▷ Salida correcta: `Consulte autoridades locales`

▷ Ejemplo de entrada: `39  N N N`
▷ Salida correcta: `Cuarentena en su casa`

Finalidad: Planteamiento de una estructura condicional anidada. Dificultad Media.

Finalidad: Separación de E/S y C. Dificultad Baja.

16. [Velocidad imputada (con un tipo enumerado)] Continuamos trabajando sobre la base del ejercicio II.11 [Velocidad imputada] de esta Relación de Problemas.

    Añada un nuevo tipo de radar, a saber de tramo. Este tipo de radares toman una foto al vehículo en un punto de la vía y luego vuelven a tomar otra foto en un punto posterior. La velocidad captada es realmente la velocidad media entre ambos puntos, pero a efectos de nuestro ejemplo es simplemente un valor más de velocidad.

Utilice el carácter 'T' para designar en la entrada de datos de este tipo de radar. En este nuevo tipo de radar, supondremos que hay un margen de 3 km/h cuando la velocidad es menor o igual que 100 y un margen del 3 % en el caso de que la velocidad sea mayor que 100.

Use una variable de tipo de dato enumerado para detectar el tipo de radar (fijo, móvil o de tramo)

▷ Ejemplo de entrada: `F 95` ▷ Salida correcta: `90`

▷ Ejemplo de entrada: `M 95` ▷ Salida correcta: `88`

▷ Ejemplo de entrada: `T 105` ▷ Salida correcta: `101.85`

Finalidad: Separación de E/S y C. Usar el tipo enumerado para detectar cuándo se produce una situación determinada. Dificultad Baja.

17. [Cambia capitalización usando un enumerado] Modifique la solución al ejercicio II.9 [Cambia capitalización] cuyo código puede descargar del siguiente enlace:

    https://decsai.ugr.es/jccubero/FP/II_CambiaCapitalizacion.cpp

    para que, dependiendo de cómo era la letra introducida, imprima en pantalla alguno de los siguientes mensajes:

    - `La letra era una mayúscula. Una vez convertida es` ...
    - `La letra era una minúscula. Una vez convertida es` ...
    - `El carácter no era una letra.`

    Hágalo separando entradas y salidas de los cómputos. Para ello, utilice una variable de tipo enumerado que represente las opciones de que un carácter sea una mayúscula, una minúscula o un carácter no alfabético.

    ¿Cuál sería el inconveniente de usar dos variables de tipo `bool`?

    Finalidad: Separación de E/S y C. Usar el tipo enumerado para detectar cuándo se produce una situación determinada. Dificultad Media.

Problemas sobre bucles

18. [Divisores de un entero] Realice un programa que lea desde teclado un entero e imprima en pantalla todos sus divisores positivos propios. Recuerde que los divisores propios de un entero n son los divisores de n distintos de 1 y el mismo n. Para obtener los divisores, basta recorrer todos los enteros menores que el valor introducido y comprobar si lo dividen. A continuación, mejorar el ejercicio para que la lectura del entero se realice usando un filtro con un bucle post test (`do while`), obligando a que sea positivo.

    En este ejercicio, como en muchos otros de esta Relación de Problemas, tendrá que mezclar entradas y salidas de datos con los cómputos. En el tema III, cuando se vean los vectores, podrá separar ambas tareas.

    ▷ Ejemplo de entrada: `16` ▷ Salida correcta: `2, 4, 8`

    Finalidad: Plantear un ejemplo sencillo de bucle y de filtro de entrada de datos. Dificultad Baja.

19. [Intervalo: Lectura de valores] Recupere la solución del ejercicio II.14 [Intervalo] de esta Relación de Problemas. Después de haber leído los datos que definen el intervalo el programa debe leer el valor de un entero `n` y a continuación debe leer `n` valores reales. Por cada uno de ellos, el programa nos dirá si pertenece o no al intervalo anterior.

    ▷ Ejemplo de entrada: `(3.5 , 5.1]  2  4.8  8.3`
    ▷ Salida correcta:

    ```
    El valor 4.8 está dentro del intervalo (3.5, 5.1]
    El valor 8.3 no está dentro del intervalo (3.5, 5.1]
    ```

    Finalidad: Plantear una estructura repetitiva básica con un condicional dentro. Dificultad Baja.

20. [Valores de la Gaussiana] Recupere la solución del ejercicio I.18 [Gaussiana] de la Relación de Problema I. Si lo desea, puede usar el código disponible en el siguiente enlace:

    https://decsai.ugr.es/jccubero/FP/I_Gaussiana.cpp

    Se pide construir un programa para imprimir el resultado de aplicar dicha función a varios valores de abscisas.

    En primer lugar, se leerán los parámetros que definen la función, es decir, la esperanza y la desviación. La esperanza puede ser cualquier valor, pero para leer el valor de desviación debe utilizar un bucle y obligar al usuario a que introduzca una valor mayor o igual que cero.

    A continuación el programa pedirá un valor `minimo`, un valor `maximo` y un `incremento`. El valor `maximo` ha de leerse con un filtro de entrada obligando a que sea mayor que

`minimo`. El programa mostrará el valor de la función gaussiana en todos los valores de x (la abscisa) entre `minimo` y `maximo` a saltos de `incremento`, es decir, `minimo`, `minimo + incremento`, `minimo + 2*incremento`, $\cdots$, hasta llegar, como mucho, a `maximo`. El incremento ha de leerse con un filtro de entrada para obligar a que sea estrictamente positivo. El siguiente ejemplo muestra el orden en la entrada de datos junto con las restricciones correspondientes:

```
12      <- Esperanza  (sin restricción)
5       <- Desviación (>= 0)
2       <- Mínimo     (sin restricción)
3       <- Máximo     (>= Mínimo)
0.5     <- Incremento (> 0)
```

▷ Ejemplo de entrada: `12 5  2 3  0.5`
▷ Salida correcta: `0.0107982 0.0131232 0.01579`
▷ Ejemplo de entrada: `12 -5 5  2 3  0.5`
▷ Salida correcta: `0.0107982 0.0131232 0.01579`
▷ Ejemplo de entrada: `12 -5 5  2 1 0 3  0.5`
▷ Salida correcta: `0.0107982 0.0131232 0.01579`

Finalidad: Ejemplo básico de bucle. Dificultad Baja.

21. [Variación porcentual: lectura de varios valores] Recupere la solución del ejercicio I.5 [Variación porcentual] de la Relación de Problema I. Si lo desea, puede usar el código disponible en el siguiente enlace:

    https://decsai.ugr.es/jccubero/FP/I_VariacionPorcentual.cpp

    Modifíquelo para realizar una lectura de múltiples pares de valores (inicial1, final1, inicial2, final2, etc.). La entrada de datos se interrumpirá cuando se introduzca cualquier valor negativo en el valor inicial. Para simplificar, supondremos que si el valor inicial introducido es positivo, el usuario también introducirá un positivo como valor final.

    Por cada par de valores, el programa mostrará la correspondiente variación porcentual. En este ejercicio puede mezclar entradas de datos con salidas y cómputos, dentro del mismo bucle.

    ▷ Ejemplo de entrada: `5 6  6 5  -1`
    ▷ Salida correcta: `Variaciones porcentuales: 20%  16.6667%`

    Finalidad: Controlar la lectura de pares de valores con termminador. Dificultad Baja.

22. [Factorial y Potencia] Calcule mediante un programa en C++ la función potencia x^n, y la función factorial $n!$ con n un valor entero y x un valor real. No pueden usarse las funciones definidas en `cmath`, por lo que tendrá que implementar los cómputos con los bucles necesarios.

    El factorial de un entero n se define de la forma siguiente:

    $$0! = 1$$

$$n! = 1 \times 2 \times 3 \times \cdots n, \ \ \forall n \geq 1$$

Escriba un programa de prueba que lea un número entero n obligando a que esté en el intervalo $[1, 20]$. A continuación lea un valor real x y calcule e imprima en pantalla el factorial de n y la potencia de x elevado a n.

Finalidad: Trabajar con bucles controlados por contador. Dificultad Baja.

23. [Interés bancario (capital reinvertido)] Recupere la solución del ejercicio I.8 [Interés bancario] de la Relación de Problema I. Si lo desea, puede usar el código disponible en el siguiente enlace:

    https://decsai.ugr.es/jccubero/FP/I_InterBancario.cpp

    Supongamos ahora que se quiere reinvertir todo el dinero obtenido (el original C más los intereses producidos) en otro plazo fijo a un año y así, sucesivamente. Construya un programa para que lea el capital, el interés I y un número de años N, y calcule e imprima todo el dinero obtenido durante cada uno de los N años, suponiendo que todo lo ganado (incluido el capital original C) se reinvierte a plazo fijo durante el siguiente año.

    Para resolver este ejercicio debe plantear una estructura repetitiva. Si lo desea, puede comprobar que el resultado es correcto, comparándolo con el de la fórmula del interés compuesto que obtiene directamente el capital final acumulado:

    $$C * (1 + I/100)^N$$

    ▷ Ejemplo de entrada: 300 5.4 3
    ▷ Salida correcta:

```
Capital obtenido transcurrido el año número 0 = 316.2
Capital obtenido transcurrido el año número 1 = 333.275
Capital obtenido transcurrido el año número 2 = 351.272
```

    Finalidad: Usar una variable acumuladora dentro del cuerpo de un bucle (aparecerá a la izquierda y a la derecha de una asignación). Dificultad Baja.

24. [Interés bancario (doblar)] Recupere la solución del ejercicio I.8 [Interés bancario] de la Relación de Problema I. Si lo desea, puede usar el código disponible en el siguiente enlace:

    https://decsai.ugr.es/jccubero/FP/I_InterBancario.cpp

    Construya un programa para calcular cuántos años han de pasar hasta llegar a doblar, como mínimo, el capital inicial. Los datos que han de leerse desde teclado son el capital inicial y el interés anual.

    ▷ Ejemplo de entrada: 300 5.4
    ▷ Salida correcta:

```
Para al menos doblar la cantidad inicial han de pasar 14 años
```

Finalidad: Usar la variable acumuladora en la misma condición del bucle. Dificultad Baja.

25. [Número Narcisista] Un número entero de n dígitos se dice que es **narcisista** si se puede obtener como la suma de las potencias n-ésimas de cada uno de sus dígitos. Por ejemplo 153 y 8208 son números narcisistas porque $153 = 1^3 + 5^3 + 3^3$ (153 tiene 3 dígitos) y $8208 = 8^4 + 2^4 + 0^4 + 8^4$ (8208 tiene 4 dígitos). Construya un programa que lea un número entero positivo y nos indique si es o no narcisista. Recuerde que para sacar el último dígito de un número entero, basta calcular el resto de la división por 10 (operador %)

    ▷ Ejemplo de entrada: `153` ▷ Salida correcta: `El número es narcisista`

    ▷ Ejemplo de entrada: `152` ▷ Salida correcta: `El número no es narcisista`

    Finalidad: Ejercitar los bucles. Dificultad Media.

26. [Mínimo de varios valores] Realice un programa que lea 5 enteros desde teclado y calcule el mínimo de dichos valores (pueden ser positivos o negativos). Realice la lectura de los enteros dentro de un bucle sobre una única variable llamada `n`.

    ▷ Ejemplo de entrada: `1 3 -1 -4 7`
    ▷ Salida correcta: `Mínimo: -4`

    Una vez hecho lo anterior, implemente un código para ir leyendo valores enteros hasta introducir un cero. Debe calcular cuántos se han introducido y cual es el mínimo de dichos valores (pueden ser positivos o negativos).

    ▷ Ejemplo de entrada: 0
    ▷ Salida correcta: `Introducidos: 0. Mínimo: 0`

    ▷ Ejemplo de entrada: `1 3 -1 2 0`
    ▷ Salida correcta: `Introducidos: 4. Mínimo: -1`

    ▷ Ejemplo de entrada: `1 3 1 2 0`
    ▷ Salida correcta: `Introducidos: 4. Mínimo: 1`

    A continuación, lea valores hasta que se introduzca el cero o cualquier entero positivo. Muestre también cuántos se han introducido y cual es el mínimo de dichos valores.

    ▷ Ejemplo de entrada: `-1 -3 -2 6`
    ▷ Salida correcta: `Introducidos: 3. Mínimo: -3`

    Finalidad: Destacar la importancia de las inicializaciones antes de entrar al bucle. Ejemplo de lectura anticipada. Dificultad Baja.

27. [Velocidad imputada -lectura en bucle-] Recupere la solución del ejercicio II.11 [Velocidad imputada] de esta Relación de Problemas. Modifique el programa principal para que vaya leyendo los siguientes datos dentro de un bucle: en primer lugar, se leerá la matrícula del vehículo (en un dato de tipo `string`), a continuación el tipo de radar (dato de tipo `char`) y finalmente la velocidad del vehículo (en un dato de tipo `double`).

Supondremos que el carácter `'F'` está asociado al radar fijo y el carácter `'M'` al radar móvil. Supondremos que los datos leídos son correctos (siempre serán `'F'` o `'M'`)

El programa calculará las velocidades imputadas según se indicó en el ejercicio II.11 [Velocidad imputada] e imprimirá en pantalla la matrícula de aquel vehículo que tenga máxima velocidad imputada (también imprimirá dicha velocidad)

La lectura de los datos terminará cuando la matrícula del vehículo sea la cadena `"#"`

▷ Ejemplo de entrada: `4312BHG F 95  8342DFW F 104  4598IJG M 95  #`
▷ Salida correcta: `8342DFW  98.8`

Finalidad: Trabajar con la lectura anticipada y bucles con condicionales dentro. Dificultad Baja.

28. [Tarifa aérea con filtro de entrada de datos] Recupere la solución del ejercicio II.10 [Tarifa aérea con descuentos] de esta misma Relación de Problema. Si lo desea, puede usar el código disponible en el siguiente enlace:

    https://decsai.ugr.es/jccubero/FP/II_TarifaAereaDescuentos.cpp

    Modifíquelo para obligar al usuario a introducir valores correctos. Por lo tanto, debe usar un filtro de entrada de datos para que el número de kilómetros sea mayor o igual que cero y el número de puntos sea un valor entre 0 y 400 (máximo número posible de puntos).

    En el siguiente ejemplo, observe que los filtros de entrada de datos *rechazan* los datos `-2 -5` (el número de kms debe ser positivo) y `1300 -450` (el número de puntos ha de estar entre 0 y 400).

    ▷ Ejemplo de entrada: `-2 -5 200  1300 -450 250`
    ▷ Salida correcta: `144`

    Finalidad: Uso de bucles en filtros de entrada de datos con condiciones compuestas. Dificultad Baja.

29. [Aproximación de π por Gregory-Leibniz] En el siglo XVII el matemático alemán Gottfried Leibniz y el matemático escocés James Gregory introdujeron una forma de calcular π a través de una serie, es decir, de una suma de términos:

$$\frac{\pi}{4} = \sum_{i=0}^{\infty} \frac{(-1)^i}{2i+1} = 1 - \frac{1}{2*1+1} + \frac{1}{2*2+1} - \frac{1}{2*3+1} + \cdots$$

    Esta es una serie infinita, pues realiza la suma de infinitos términos. Como en Programación no podemos realizar un número infinito de operaciones, habrá que parar en un índice dado, llamémosle `tope`, obteniendo por tanto una aproximación al valor de π. Usaremos el símbolo $\approx$ para denotar esta aproximación:

$$\frac{\pi}{4} \approx \sum_{i=0}^{i=\texttt{tope}} \frac{(-1)^i}{2i+1} = 1 - \frac{1}{2*1+1} + \frac{1}{2*2+1} - \frac{1}{2*3+1} + \cdots + \frac{(-1)^{\texttt{tope}}}{2*\texttt{tope}+1}$$

$$= 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots + \frac{(-1)^{\texttt{tope}}}{2 * \texttt{tope} + 1}$$

Construya un programa que lea el valor `tope` obligando a que esté entre 1 y cien mil, calcule la aproximación de π mediante la anterior serie e imprima el resultado en pantalla.

Resuelva este problema de tres formas distintas (no hace falta que entregue tres ejercicios: baste con que incluya las dos primeras soluciones dentro de un comentario):

a) Use la función `pow` (potencia) de `cmath` para implementar $(-1)^i$

b) Para cada valor de i, calcule $(-1)^i$ con un bucle, tal y como hizo en el ejercicio de la potencia (problema II.22 [Factorial y Potencia])

c) De una forma más eficiente que las anteriores. Por ejemplo, observe que el valor de $(-1)^i$ es 1 para los valores pares de i y -1 para los impares

Recuerde que, para visualizar 15 cifras decimales, por ejemplo, debe incluir la sentencia `cout.precision(15);` antes de realizar la salida en pantalla.

▷ Ejemplo de entrada: `1000` ▷ Salida correcta: `3.14259165433954`

▷ Ejemplo de entrada: `100000` ▷ Salida correcta: `3.14160265348972`

Ampliación:

Es fácil demostrar que el valor absoluto de cada sumando se va haciendo cada vez más pequeño, por lo que cuantos más sumandos incluyamos, mejor será la aproximación. En cualquier caso, la serie *converge* al valor correcto de forma muy lenta. Habría que incluir cinco mil millones de términos para obtener 10 cifras decimales exactas en el cómputo de π.

Finalidad: Bucles simples. Acumulación de sumas. Dificultad Baja.

30. [Aproximación de π por Wallis] Otra aproximación de π introducida en el siglo XVII por el matemático inglés John Wallis viene dada por:

$$\frac{\pi}{2} \approx \frac{2}{1} \cdot \frac{2}{3} \cdot \frac{4}{3} \cdot \frac{4}{5} \cdot \frac{6}{5} \cdot \frac{6}{7} \cdots$$

Construya un programa que lea el valor `tope` obligando a que esté entre 1 y cien mil, calcule la aproximación de π mediante la anterior fórmula (multiplicando un total de `tope` fracciones) e imprima el resultado en pantalla.

Debe resolver este problema de dos formas distintas, a saber:

a) Observe que el numerador y el denominador varían de forma alternativa (aunque ambos de la misma forma, a saltos de 2). Cuando a uno le toca cambiar, el otro permanece igual. Este comportamiento se puede controlar con una única variable de tipo de dato `bool`.

b) Otra forma de implementar los cambios en el numerador y denominador es observando que en cada iteración, el numerador es el denominador de la iteración anterior más 1 y el denominador es el numerador de la iteración anterior más 1.

▷ Ejemplo de entrada: `1000` ▷ Salida correcta: `3.1400238186006`

▷ Ejemplo de entrada: `100000` ▷ Salida correcta: `3.14157694582286`

Nota: Dependiendo del orden de ejecución de la multiplicación y la división, es posible que pudiese variar algún decimal.

Finalidad: Bucles simples. Acumulación de productos. Usar un `bool` *fijado en la iteración anterior. Dificultad Media.*

31. [Aproximación de π por Madhava sin usar pow] En el siglo XIV el matemático indio Madhava of Sangamagrama calculó el arco tangente a través de un desarrollo de Taylor (este tipo de desarrollos se ve en la asignatura de Cálculo)

$$\arctan(x) = \sum_{i=0}^{\infty} \frac{(-1)^i x^{2i+1}}{2i+1}$$

Usando como x el valor 1, obtenemos la serie de Leibniz vista en el ejercicio II.29:

$$\arctan(1) = \frac{\pi}{4} = \sum_{i=0}^{\infty} \frac{(-1)^i}{2i+1}$$

Usando como x el valor $\frac{1}{\sqrt{3}}$, obtenemos:

$$\arctan\left(\frac{1}{\sqrt{3}}\right) = \frac{\pi}{6} = \sum_{i=0}^{\infty} \frac{(-1)^i \left(\frac{1}{\sqrt{3}}\right)^{2i+1}}{2i+1}$$

Por lo tanto, podemos usar la siguiente aproximación:

$$\frac{\pi}{6} \approx \sum_{i=0}^{\texttt{tope}} \frac{(-1)^i \left(\frac{1}{\sqrt{3}}\right)^{2i+1}}{2i+1}$$

Construya un programa que lea el valor `tope` obligando a que esté entre 1 y cien mil, calcule la aproximación de π mediante la anterior serie e imprima el resultado en pantalla. Esta serie converge mucho más rápido a π que las de los ejercicios anteriores.

Importante: En la implementación de esta solución NO puede usar la función `pow` ni ningún condicional `if`. Se le pide expresamente que para el cómputo de cada término, intente aprovechar los cómputos realizados en la iteración anterior. Por ejemplo, no debe calcular $(-1)^i$ en cada iteración con un bucle sino que debe aprovechar el valor obtenido en la iteración anterior de $(-1)^{(i-1)}$ para así calcular $(-1)^i$.

▷ Ejemplo de entrada: `10`

▷ Salida correcta: `3.14159330450308`

▷ Ejemplo de entrada: `100`

▷ Salida correcta: `3.14159265358979`

Finalidad: Bucles simples. Aprovechar cómputos de la iteración anterior. Dificultad Media.

32. [Número Reflejo] Cualquier valor entero puede asociarse a otro mediante un proceso que invierte (refleja) sus dígitos. Por ejemplo, el valor reflejado de 123 es 321 y el de -678 sería -876. Observe que el valor reflejado de 9100 y de 910 es el mismo, 19.

    Construya un programa que lea un entero `n`, calcule su reflejo y lo imprima en pantalla. Resuelva primero el problema usando la función `pow` y luego intente resolverlo de otra forma lo más eficiente posible evitando el uso de `pow`.

    Finalidad: Ejercitar los bucles. Dificultad Media.

33. [Raíz cuadrada(Wilkes)] Para calcular la raíz cuadrada de un número los procesadores de los ordenadores recurren a métodos utilizando sumatorias de términos en los que sólo aparecen operaciones sencillas como sumas, multiplicaciones y divisiones.

    En este ejercicio, vamos a usar una aproximación introducida en los años cincuenta por Wilkes, Wheeler y Gill. Estos autores definieron el siguiente método para calcular la raíz cuadrada de un número real $r \in (0, 3)$ pero se puede extender para calcular la raíz de cualquier número (consulte https://www.quora.com/How-do-computers-calculate-square-roots):

    Inicialización:

    $$a_0 = r$$
    $$c_0 = r - 1$$

    Vaya repitiendo el cómputo de los siguientes términos hasta que c_n se aproxime suficientemente a cero: pararemos las iteraciones cuando el valor absoluto de c_{n+1} sea menor que una constante `EPSILON = 1e-15`.

    $$a_{n+1} = a_n - a_n c_n/2$$
    $$c_{n+1} = c_n^2(c_n - 3)/4$$

    Construya un programa que lea un real r obligando a que esté en el intervalo $(0, 3)$ e imprima en pantalla el resultado de aplicar el anterior algoritmo. Imprima también el valor de la raíz cuadrada calculado usando directamente la función `sqrt` de `cmath` para así poder comparar el resultado obtenido. Imprima también el número de iteraciones que se han realizado.

    Si lo desea, puede usar el esbozo del programa que puede encontrar en el siguiente enlace:

https://decsai.ugr.es/jccubero/FP/II_RaizCuadradaEsbozo.cpp

▷ Ejemplo de entrada: 2.1

▷ Salida correcta:

```
Raíz cuadrada de 2.1 calculada con sqrt: 1.44913767461894
Aproximación de la raíz cuadrada de 2.1: 1.44913767461894
Número de iteraciones: 7
```

Finalidad: Bucles simples. Dificultad Baja.

34. [Factorial y Potencia: bucle `for`] Retome la solución del ejercicio II.22 [Factorial y Potencia] y modifíquelo para que el bucle principal sea un bucle `for` en vez de un bucle `while`.

    Finalidad: Ejercitar los bucles `for`. *Dificultad Baja.*

35. [Pirámide] Cree un programa que ofrezca en pantalla la siguiente salida:

```
1 2 3 4 5 6
2 3 4 5 6
3 4 5 6
4 5 6
5 6
6
```

    Modifique la solución para que se lea desde teclado el valor inicial y el número de filas a imprimir. En el ejemplo anterior, se supone que el usuario ha introducido el 1 (valor inicial) y el 6 (número de filas).

    Finalidad: Ejercitar los bucles anidados. Dificultad Baja.

36. [Cuadrado] Cree un programa que ofrezca en pantalla la siguiente salida:

```
1 2 3 4 5 6
2 3 4 5 6 7
3 4 5 6 7 8
4 5 6 7 8 9
5 6 7 8 9 10
6 7 8 9 10 11
```

    Modifique la solución para que se lea desde teclado el valor inicial y el número de filas a imprimir. En el ejemplo anterior, se supone que el usuario ha introducido el 1 (valor inicial) y el 6 (número de filas).

    Finalidad: Ejercitar los bucles anidados. Dificultad Media.

37. [Divisores anidado] Retome la solución del ejercicio II.18 [Divisores de un entero] de esta Relación de Problemas. Modifíquelo para que el programa principal lea un número entero positivo `tope` e imprima en pantalla los divisores de todos y cada uno de los

números positivos menor o iguales que `tope`

▷ Ejemplo de entrada: 6

▷ Salida correcta:

```
Divisores de 6: 2  3
Divisores de 5: Ninguno
Divisores de 4: 2
Divisores de 3: Ninguno
Divisores de 2: Ninguno
```

Finalidad: Ejercitar los bucles anidados. Dificultad Baja.

38. [Interés bancario anidado] Sobre la solución del ejercicio II.23 [Interés bancario (capital reinvertido)] de esta relación de problemas, se pide lo siguiente. Supondremos que sólo pueden introducirse intereses enteros (`1, 2, 3`, etc). Se pide calcular el capital obtenido al término de cada año, pero realizando los cálculos para todos los tipos de interés enteros menores o iguales que el introducido (en pasos de 1). Por ejemplo, si el usuario introduce un interés igual a 5 y un número de años igual a 3, hay que mostrar el capital ganado al término de cada uno de los tres años a un interés del 1 %, a continuación, lo mismo para un interés del 2 % y así sucesivamente hasta llegar al 5 %. El programa debe mostrar una salida del tipo:

```
Cálculos realizados al 1%:

Dinero obtenido en el año número 1 = 2020
Dinero obtenido en el año número 2 = 2040.2
Dinero obtenido en el año número 3 = 2060.6

Cálculos realizados al 2%:

Dinero obtenido en el año número 1 = 2040
Dinero obtenido en el año número 2 = 2080.8
Dinero obtenido en el año número 3 = 2122.42
.......
```

Finalidad: Empezar a trabajar con bucles anidados. Dificultad Baja.

39. [Tarifa aérea: múltiples billetes] Recupere la solución del ejercicio II.28 [Tarifa aérea con filtro de entrada de datos] de esta misma Relación de Problema. Si lo desea, puede usar el código disponible en el siguiente enlace:

    https://decsai.ugr.es/jccubero/FP/II_TarifaAereaFiltroEntrada.cpp

    Modifique dicha solución para que el programa calcule la tarifa final de una serie de billetes. Para ello, cada vez que se introduzcan los datos de un nuevo billete, el usuario

introducirá el carácter 'N' (Nuevo). La entrada de datos finaliza con el carácter '#'. Si se introduce un carácter distinto, el programa pedirá un nuevo carácter.

Recuerde que el primer dato a leer era el número de kilómetros del trayecto y el segundo dato corresponde al número de puntos del cliente. En el ejercicio II.28 [Tarifa aérea con filtro de entrada de datos] ya se había programado el filtro de entrada para los datos de los kilómetros y número de puntos.

En el siguiente ejemplo, observe que el nuevo filtro de entrada de datos *rechaza* los caracteres `J K` ya que no son ni `N` ni #.

▷ Ejemplo de entrada:

```
J K N -2 -5 200    1300 -450 250     P N 650    1300 -450 90 #
```

▷ Salida correcta: `144  185`

Finalidad: Filtros de entrada de datos. Bucles anidados. Lectura de datos con terminador. Dificultad Baja.

40. [Collatz] *(Examen Ordinario 2025)* La sucesión de *Collatz* a partir de cualquier número N $\in \mathbb{Z}^+$ es una secuencia en la que cada nuevo término de la sucesión se genera a partir del anterior de acuerdo a la siguiente regla:

    - Si el número es par, se divide entre 2.
    - Si el número es impar, se multiplica por 3 y se suma 1.

    La conjetura de Collatz establece que esta sucesión termina siempre en 1. Por ejemplo, la sucesión de Collatz a partir de $N = 18$ sería la siguiente: *18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1*. Se denomina *órbita* al número de pasos que se ha dado hasta llegar a 1, es decir, la longitud de la sucesión menos 1. En el ejemplo anterior, el número 18 tiene una órbita de 20.

    Suponiendo que la conjetura es cierta, se pide que desarrolle un programa que calcule para un intervalo `[min, max]` el número que tiene una mayor órbita. Por ejemplo, para el intervalo $[5, 8]$ el 7 es el entero que tiene la mayor órbita (16).

    Construya un programa principal que lea un entero `min`, otro entero `max`, calcule el entero del intervalo `[min,max]` con mayor órbita y muestre en pantalla dicho entero junto con su órbita.

    ▷ Ejemplo de entrada: `5  8` ▷ Salida correcta: `7  16`

    ▷ Ejemplo de entrada: `2  12` ▷ Salida correcta: `9  19`

    Finalidad: Ejercitar los bucles anidados. Dificultad Media.

41. [Mayor nota media] *(Examen Prácticas Noviembre 2019)*

    Se quiere calcular la máxima nota media de evaluación continua de un conjunto de alumnos. Para ello, se anota en un fichero un número entero con el código del alumno y las notas que ha conseguido. El número de notas puede variar de un alumno a otro, por lo

que se terminará la introducción de las notas con un -1. La entrada de datos finaliza con el código de alumno 0.

Cree un programa que lea las notas desde la entrada por defecto, y calcule el alumno con mayor nota media. Puede suponer que los datos de entrada son siempre correctos.

Por ejemplo, para el siguiente registro de entradas, el alumno con máxima nota es el que tiene identificador 17 con una nota media de 9.5

```
11    8 7 6 -1
14    3 -1
 7    9 9 8 7 -1
17    10 9 -1
 8    9 9 -1
15    6 7 5 -1
 5    8 -1
 0
```

El programa imprimirá únicamente los dos enteros (código del mejor alumno y su nota media):

```
17  9.5
```

Finalidad: Bucles anidados. Lectura de datos con terminador. Dificultad Baja.

42. [Números perfectos] *(Examen Prácticas Noviembre 2019)*

Diremos que un número $n > 1$ es *perfecto* si es igual a la suma de todos sus divisores positivos (incluyendo el 1) excepto él mismo. Por ejemplo, el 6 es perfecto ya que sus divisores son 1, 2 y 3 y se cumple que 6=1+2+3.

Construya un programa principal que lea un entero cualquiera `min` desde teclado obligando a que sea mayor o igual que 1 y otro entero `max` obligando a que sea mayor o igual que `min`. El programa debe mostrar todos los números perfectos que hay entre `min` y `max`.

Por ejemplo, si `min = 13` y `max = 500`, el programa debe imprimir `28  496`.

En este programa no se preocupe por mezclar entradas y salidas de datos con cómputos dentro del mismo bucle.

Finalidad: Ejercitar los bucles anidados. Dificultad Media.

43. [Muy divisible] *(Examen Prácticas Noviembre 2019)* Diremos que un número entero n es k*-divisible* si n tiene k o más divisores propios (no se consideran ni el 1 ni el propio valor).

Construya un programa que lea un entero cualquiera `min` desde teclado. A continuación debe leer otro entero `max` obligando, con un filtro de entrada de datos, a que sea mayor o igual que `min`. Finalmente el programa leerá un valor entero k con otro filtro para que sea mayor o igual que 1.

El programa imprimirá por pantalla aquellos números n en el rango `[min, max]` que tienen k o más divisores (es decir, los números k-*divisibles*).

Por ejemplo, si `min` = 78, `max` = 90 y $k = 3$, el programa debe imprimir `78 80 81 84 88 90` ya que esos son los números en el rango `[78, 90]` que tienen 3 o más divisores.

En este ejercicio no podrá separar entradas/salidas de los cómputos, por lo que dentro del bucle principal tendrá que hacer ambas tareas.

▷ Ejemplo de entrada: `78 90 3`

▷ Salida correcta: `78 80 81 84 88 90`

Finalidad: Ejercitar los bucles anidados. *Dificultad Media.*

Ejercicios adicionales (estructura de control condicional)

44. [Media y desviación típica] Amplíe el ejercicio I.20 [Media y desviación] de la relación de problemas I, para que, una vez calculada la media y la desviación, el programa imprima por cada uno de los valores introducidos previamente, si está por encima o por debajo de la media. Por ejemplo:

```
33 es menor que su media
48 es mayor o igual que su media
.....
```

Nota. Los valores introducidos son enteros, pero la media y la desviación son reales.

Finalidad: Plantear un ejemplo básico con varias estructuras condicionales dobles consecutivas. *Dificultad Baja.*

45. [Multa autovía] La Dirección General de Tráfico publica en su página web las sanciones a aplicar por infracción de velocidad. Si lo desea, se pueden consultar en el siguiente enlace (aunque no es necesario para resolver este ejercicio):

https://www.dgt.es/nuestros-servicios/multas-y-sanciones/conoce-los-tipos-de-infracciones-y-sanciones/

Queremos determinar la sanción a aplicar en una autovía, cuyo límite de velocidad es 120. En la siguiente tabla se muestra la velocidad del vehículo y la sanción correspondiente (número de puntos del carnet de conducir que se restan y la multa en euros)

```
121 -> 0 puntos, 100 euros
151 -> 2 puntos, 300 euros
171 -> 4 puntos, 400 euros
181 -> 6 puntos, 500 euros
191 -> 6 puntos, 600 euros
```

Escriba un programa que lea la velocidad del vehículo e imprima en pantalla la sanción correspondiente (número de puntos a detraer y multa en euros)

Finalidad: Planteamiento de una estructura condicional anidada. *Dificultad Baja.*

46. [Secuencia de temperaturas] Construya un programa que calcule cuándo se produjo la mayor secuencia de días consecutivos con temperaturas crecientes. El programa leerá una secuencia de reales representando temperaturas. Una temperatura es correcta si su valor está en el intervalo `[-90, 60]` (los extremos representan las temperaturas extremas registradas en la Tierra). La entrada de datos terminará cuando se introduzca una temperatura fuera del rango anterior. El programa debe calcular la subsecuencia de números ordenada, de menor a mayor, de mayor longitud.

El programa nos debe decir la posición donde comienza la subsecuencia y su longitud.

Por ejemplo, ante la entrada siguiente:

`17.2 17.3 16.2 16.4 17.1 19.2 18.9 100`

el programa nos debe indicar que la mayor subsecuencia empieza en la posición 3 (en el `16.2`) y tiene longitud 4 (termina en `19.2`)

Considere los siguientes consejos:

- Tendrá que leer sobre dos variables `anterior` y `actual`, para así poder comparar el valor actual con el anterior.
- Se recomienda que use la técnica de lectura anticipada, por lo que tendrá que leer un primer valor y comprobar si está en el rango adecuado:

```
cin >> anterior;
final_entrada_datos = anterior < MIN_TEMP
                      ||
                      anterior > MAX_TEMP;

while (! final_entrada_datos){
   cin >> actual;
   ......
}
```

  Dentro del cuerpo del bucle tendrá que comparar el valor `actual` con los extremos del rango de temperaturas, tal y como se hizo antes de entrar al bucle con el valor `anterior`. Esto hace que repitamos un código muy parecido. Lo resolveremos cuando veamos las funciones.

▷ Ejemplo de entrada: `17.2 17.3 16.2 16.4 17.1 19.2 18.9 100`
▷ Salida correcta: `Inicio: 3  Longitud: 4`
▷ Ejemplo de entrada: `17.2 17.3 16.2 16.4 17.1 19.2 100`
▷ Salida correcta: `Inicio: 3  Longitud: 4`
▷ Ejemplo de entrada: `17.2 17.3 100`
▷ Salida correcta: `Inicio: 1  Longitud: 2`
▷ Ejemplo de entrada: `17.2 15.3 100`
▷ Salida correcta: `Inicio: 1  Longitud: 1`
▷ Ejemplo de entrada: `17.2 100`
▷ Salida correcta: `Inicio: 1  Longitud: 1`
▷ Ejemplo de entrada: `100`
▷ Salida correcta: `Inicio: 1  Longitud: 0`

Finalidad: Trabajar con bucles que comparan un valor actual con otro anterior. Dificultad Media.

47. [Coordenadas geográficas (distancia)] Recupere la solución del ejercicio I.42 [Coorde-

nadas geográficas (distancia)] de la Relación de Problema I. Si lo desea, puede usar el código disponible en el siguiente enlace:

https://decsai.ugr.es/jccubero/FP/I_CoordenadasGeograficas.cpp

Re-escríbalo sin utilizar la función `min` (elimine por tanto la inclusión de la biblioteca `algorithm`). Tendrá que calcular el mínimo de dos valores con un condicional simple.

Finalidad: Plantear una estructura condicional sencilla. Dificultad Baja.

48. [Codificación de caracteres con algoritmo de rotación. Condicional simple] Reutilice la solución del ejercicio I.40 [Codificación de caracteres con algoritmo de rotación] y cambie la implementación para realizar la codificación del carácter utilizando un condicional simple en vez de una expresión tan compleja como la que se utilizó en dicho ejercicio. En definitiva, en este ejercicio no puede usar el operador módulo %, aunque sí puede usar condicionales simples.

    Una vez obtenida la letra cifrada, descífrela para comprobar que se obtiene la letra de partida.

    En este ejercicio puede suponer que la letra introducida será una mayúscula y que el desplazamiento no será mayor que el número de letras mayúsculas.

    ▷ Ejemplo de entrada: `4 A` ▷ Salida correcta: `E A`

    ▷ Ejemplo de entrada: `4 Y` ▷ Salida correcta: `C Y`

    Finalidad: Plantear una estructura condicional simple. Actualizar una variable según una condición. Dificultad Baja.

49. [Codificación de caracteres con algoritmo de rotación. Desplazamiento arbitrario] Reutilice la solución del ejercicio II.48 [Codificación de caracteres con algoritmo de rotación. Condicional simple] y modifíquela para que tenga en cuenta desplazamientos de cualquier tamaño. Ahora sí puede volver a usar el operador módulo (%)

    En este ejercicio puede suponer que la letra introducida será una mayúscula.

    ▷ Ejemplo de entrada: `4 A` ▷ Salida correcta: `E A`

    ▷ Ejemplo de entrada: `30 A` ▷ Salida correcta: `E A`

    ▷ Ejemplo de entrada: `56 A` ▷ Salida correcta: `E A`

    ▷ Ejemplo de entrada: `4 Y` ▷ Salida correcta: `C Y`

    Finalidad: Plantear una estructura condicional simple. Actualizar una variable según una condición. Dificultad Baja.

50. [Mayoría absoluta] Se quiere construir un programa que calcule las posibles coaliciones de dos partidos que se pueden realizar para alcanzar la mayoría absoluta después de una votación en la que únicamente participan 3 partidos. Cuando se vea el tema de vectores se ampliará este ejercicio para tratar con un número cualquiera de partidos.

    El programa leerá el número de votos presenciales que ha obtenido el primer partido (llamémosle A) y a continuación leerá el número de votos por correo obtenidos por el

mismo partido. A continuación se leerán los mismos datos de otros dos partidos más (llamémosles B y C). El programa debe mostrar si hay algún partido que ha alcanzado la mayoría absoluta, es decir, que el total de votos obtenidos (los presenciales más los obtenidos por correo) sea mayor estricto que la mitad de la suma total de votos de todos los partidos. En el caso de que ningún partido haya alcanzado la mayoría absoluta, el programa debe mostrar las coaliciones de dos partidos que lleven a una mayoría absoluta. Debe tener en cuenta que puede haber más de una posible coalición que obtenga dicha mayoría absoluta.

Si lo desea, puede usar el esbozo del programa que puede encontrar en el siguiente enlace:
https://decsai.ugr.es/jccubero/FP/II_MayoriaAbsolutaEsbozo.cpp

Por simplificar, se utilizarán enteros pequeños en los casos de prueba:

▷ Ejemplo de entrada: `2 4  3 3  12 13`
▷ Salida correcta: `Mayoría absoluta: C`

▷ Ejemplo de entrada: `2 4  3 3  5 1`
▷ Salida correcta: `Mayoría absoluta: AB AC BC`

▷ Ejemplo de entrada: `0 0  0 0  0 0`
▷ Salida correcta: `Mayoría absoluta: Error en los datos`

▷ Ejemplo de entrada: `1 0  2 2  2 1`
▷ Salida correcta: `Mayoría absoluta: AB BC`

Finalidad: Analizar si se han de usar estructruras condicionales anidadas o consecutivas. Dificultad Baja.

51. Queremos gestionar la nómina de los empleados de un centro de atención telefónica. Construya un programa que lea el salario por hora (dato de tipo real) de un empleado, el número de horas trabajadas durante el mes actual (dato de tipo entero) el número de casos resueltos de forma satisfactoria (dato de tipo entero) y el grado medio de satisfacción de los usuarios de los servicios telefónicos con el empleado en cuestión (real entre 0 y 5).

Se quiere aplicar una subida salarial en función de varios factores. En ejercicios sucesivos se irán planteando distintas posibilidades. La primera que se quiere implementar es la siguiente:

Se aplicará una subida del 4% a los empleados que han resuelto más de 30 casos.

`Más de 30 casos resueltos:`	`+4%`

Imprima el salario final en pantalla.

▷ Ejemplo de entrada: `8.5 150 32 5` ▷ Salida correcta: `1326`

▷ Ejemplo de entrada: `7.5 130 24 3` ▷ Salida correcta: `975`

Finalidad: Plantear una estructura condicional de actualización de una variable. Dificultad Baja.

52. Recupere la solución del ejercicio II.51 sobre el cómputo de la nómina de los trabajadores de un centro de atención telefónica. Implemente ahora el siguiente criterio para la subida salarial. Se aplicará una subida del 4% a los empleados que han resuelto más de 30 casos y una subida del 2% si el grado de satisfacción media de los usuarios es mayor o igual que `4.0`. Ambas subidas son compatibles, es decir, si un trabajador cumple las dos condiciones, se le aplicarán ambas subidas.

    Resuelva este ejercicio considerando que la nueva subida del 2% se realiza sobre el salario inicial y no sobre el resultado de haber aplicado, en su caso, la otra subida del 4%.

    ```
    Más de 30 casos resueltos:                    +4%
    Grado de satisfacción >= 4:                   +2%
    ```

    ▷ Ejemplo de entrada: `8.5 150 32 5` ▷ Salida correcta: `1351.5`
    ▷ Ejemplo de entrada: `8.5 150 29 5` ▷ Salida correcta: `1300.5`
    ▷ Ejemplo de entrada: `7.5 130 24 3` ▷ Salida correcta: `975`

    Finalidad: Plantear estructuras condicionales consecutivas. Dificultad Baja.

53. Escriba un programa en C++ para que lea tres enteros desde teclado y nos diga si están ordenados (da igual si es de forma ascendente o descendente) o no lo están. Por ejemplo, la sucesión de números `3, 6, 9` estaría ordenada así como la serie `13, 2, 1` pero no lo estaría la serie `3, 9, 5`.

    Finalidad: Plantear una estructura condicional doble con una expresión lógica compuesta. Dificultad Baja.

54. [Tres valores ordenados separando E/S y C con un enumerado] Modifique el ejercicio II.53 para que el programa nos diga si los tres valores leídos están ordenados de forma ascendente, ordenados de forma descendente o no están ordenados. Para resolver este problema, debe usar una variable de tipo enumerado.

    Finalidad: Separación de E/S y C. Usar el tipo enumerado para detectar cuándo se produce una situación determinada. Dificultad Baja.

55. Modifique la solución del ejercicio II.52 para que ambas subidas salariales sean excluyentes, es decir, si se aplica una, no se aplicará la otra. En el caso de que ambas sean aplicables, debe aplicarse la subida más ventajosa para el trabajador, es decir, la del 4%.

    ```
    De forma exclusiva:

    Más de 30 casos resueltos:                    +4%
    Grado de satisfacción >= 4:                   +2%
    ```

    ▷ Ejemplo de entrada: `8.5 150 32 5` ▷ Salida correcta:
    ▷ Ejemplo de entrada: `8.5 150 29 5` ▷ Salida correcta:

▷ Ejemplo de entrada: `7.5 130 24 3` ▷ Salida correcta:

Finalidad: Plantear estructuras condicionales anidadas. Dificultad Baja.

56. La tabla para el cálculo del precio a pagar en los parkings de Madrid para el 2015 es la siguiente:

    Si permanece más de 660 minutos se paga una única tarifa de 31.55 euros
    Desde el minuto 0 al 30: 0.0412 euros cada minuto
    Desde el minuto 31 al 90: 0.0370 euros cada minuto
    Desde el minuto 91 al 120: 0.0311 euros cada minuto
    Desde el minuto 121 al 660: 0.0305 euros cada minuto

    Dado un tiempo de entrada (hora, minuto y segundo) y un tiempo de salida, construya un programa que calcule la tarifa final a cobrar. Para calcular el número de minutos entre los dos instantes de tiempo, puede utilizar la solución del ejercicio I.14 [Segundos entre dos instantes] de la Relación de Problemas I.

    ▷ Ejemplo de entrada: `2 1 30 2 1 29` ▷ Salida correcta: `-1`
    ▷ Ejemplo de entrada: `2 1 30 2 1 31` ▷ Salida correcta: `0`
    ▷ Ejemplo de entrada: `2 1 30 2 2 31` ▷ Salida correcta: `0.0412`
    ▷ Ejemplo de entrada: `2 1 30 2 41 31` ▷ Salida correcta: `1.606`
    ▷ Ejemplo de entrada: `2 1 30 3 41 31` ▷ Salida correcta: `3.767`
    ▷ Ejemplo de entrada: `2 1 30 5 41 31` ▷ Salida correcta: `7.439`
    ▷ Ejemplo de entrada: `2 1 30 23 1 1` ▷ Salida correcta: `31.55`

    Finalidad: Plantear una estructura condicional anidada. Dificultad Baja.

57. Modifique la solución del ejercicio II.52 para que también aplique una subida del 3% a los que han resuelto entre 20 y 30 casos:

    ```
    Entre 20 y 30 casos resueltos:                  +3%
    Más de 30 casos resueltos:                      +4%

    Grado de satisfacción >= 4:                     +2%
    ```

    ▷ Ejemplo de entrada: `8.5 150 32 5` ▷ Salida correcta: `1351.5`
    ▷ Ejemplo de entrada: `7.5 130 24 3` ▷ Salida correcta: `1004.25`
    ▷ Ejemplo de entrada: `7.5 130 24 4` ▷ Salida correcta: `1023.75`

    Finalidad: Plantear una estructura condicional anidada. Dificultad Baja.

58. Construya un programa para calcular el importe total a facturar de un pedido. El programa leerá el número de unidades vendidas y el precio de venta de cada unidad. Si la cantidad vendida es mayor de 100 unidades, se le aplica un descuento del 3 %. Por otra parte, si el precio final de la venta es mayor de 700 euros, se aplica un descuento del 2 %. Ambos descuentos son acumulables. Obtenga el importe final e imprímalo en pantalla.

Vamos a cambiar el criterio de los descuentos. Supondremos que sólo se aplicará el descuento del 2 % (por una venta mayor de 700 euros) cuando se hayan vendido más de 100 unidades, es decir, para ventas de menos de 100 unidades no se aplica el descuento del 2 % aunque el importe sea mayor de 700 euros.

Cambiar el programa para incorporar este nuevo criterio.

Finalidad: Plantear una estructura condicional anidada. Dificultad Baja.

59. Modifique la solución del ejercicio II.53 (valores ordenados) para que no se mezclen E/S y C (entradas/salidas y cómputos) dentro de la misma estructura condicional.

    Finalidad: Diseñar programas que separen Entradas/Salidas y cómputos. Dificultad Baja.

60. Modifique la solución del ejercicio II.3 (año bisiesto) para que no se mezclen E/S y C (entradas/salidas y cómputos) dentro de la misma estructura condicional.

    Finalidad: Diseñar programas que separen Entradas/Salidas y cómputos. Dificultad Baja.

61. Cree un programa que lea los datos fiscales da una persona, reajuste su renta bruta según el criterio que se indica posteriormente e imprima su renta neta final.

    - La renta bruta es la cantidad de dinero íntegra que el trabajador gana.
    - La retención fiscal es el tanto por ciento que el gobierno *se queda*.
    - La renta neta es la cantidad que le queda al trabajador después de quitarle el porcentaje de retención fiscal, es decir:
      Renta_neta = Renta_bruta - Renta_bruta * Retención final / 100

    Los datos a leer son:

    - Si la persona es un trabajador autónomo o no
    - Si es pensionista o no
    - Estado civil
    - Renta bruta (total de ingresos obtenidos)
    - Retención inicial a aplicar.

    La retención inicial se va a modificar ahora atendiendo al siguiente criterio:

    - Se baja 3 puntos la retención fiscal a los autónomos, es decir, si la retención inicial era de un 15 %, por ejemplo, la retención final a aplicar será de un 12 % (por lo que la renta neta final será mayor)
    - Para los no autónomos:
      - Se sube un punto la retención fiscal a todos los pensionistas, es decir, si la retención

inicial era de un 13 %, por ejemplo, la retención final a aplicar será de un 14 % (por lo que la renta neta final será menor)

- Al resto de trabajadores (no autónomo y no pensionista) se le aplica a todos una primera subida lineal de dos puntos en la retención inicial.

  Una vez hecha esta subida, se le aplica (sobre el resultado anterior) las siguientes subidas **adicionales**, dependiendo de su estado civil y niveles de ingresos:

  - Se sube otros dos puntos la retención fiscal si la renta bruta es menor de 20.000 euros
  - Se sube otros 2.5 puntos la retención fiscal a los casados con renta bruta superior a 20.000 euros
  - Se sube otros tres puntos la retención fiscal a los solteros con renta bruta superior a 20.000 euros

Una vez calculada la retención final, habrá que aplicarla sobre la renta bruta para así obtener la renta final del trabajador.

Finalidad: Plantear una estructura condicional anidada. *Dificultad Media.*

Ejercicios adicionales (bucles)

62. [Mayúscula a minúscula con filtro de entrada] Se pide leer un carácter desde teclado, obligando al usuario a que sea una letra mayúscula. Para ello, habrá que usar una estructura repetitiva `do while`, de forma que si el usuario introduce un carácter que no sea una letra mayúscula, se le volverá a pedir otro carácter. Calcule la minúscula correspondiente e imprímala en pantalla. No pueden usarse las funciones `tolower` ni `toupper`de la biblioteca `cctype`.

    Finalidad: Trabajar con bucles con condiciones compuestas. Dificultad Baja.

63. [Lectura en rango] Se pide leer dos enteros `min` y `max` que representarán un rango de valores `[min,max]`. El primer valor a leer, `min`, debe ser un número positivo y el segundo valor `max`, debe ser mayor que `min`. El programa irá leyendo estos dos valores hasta que el usuario los introduzca correctamente.

    Una vez leídos ambos valores, el programa pedirá otro entero `nuevo` obligando a que esté dentro del intervalo `[min, max]`. Si el usuario introduce más de 3 valores fuera del rango, el bucle terminará y se mostrará en pantalla un mensaje indicando que superó el número de intentos máximo. En caso contrario, es decir, en el caso de que el usuario introduzca un valor en el rango pedido, el bucle también terminará y se mostrará en pantalla el resultado de calcular `nuevo - min` y `max - nuevo`.

    ▷ Ejemplo de entrada: `-5 -6 -7 -1  5  3 4 2  8  7`
    ▷ Salida correcta: `2 1`
    ▷ Ejemplo de entrada: `-5 -6 -7 -1  5  3 4 2  8  4 9 7`
    ▷ Salida correcta: `2 1`
    ▷ Ejemplo de entrada: `-5 -6 -7 -1  5  3 4 2  8  4 9 10`
    ▷ Salida correcta: `Número de intentos sobrepasado`

    Finalidad: Trabajar con bucles con condiciones compuestas en filtros de entrada de datos. Dificultad Media.

64. [Diferencia entre instantes con filtro de entrada] Amplíe el ejercicio I.14 [Segundos entre dos instantes] de la Relación de Problemas I de manera que se obligue al usuario a que el segundo instante introducido sea posterior al primero. Suponemos que ambos instantes corresponden al mismo día. Para comparar los instantes puede usar la solución del ejercicio II.13 [Comparación de dos instantes] de esta Relación de Problemas.

    Finalidad: Trabajar con condicionales complejos y filtros de entradas de datos. Reutilizar código ya escrito y verificado. Dificultad Media.

65. [Número feliz] Se dice que un número natural es feliz si cumple que si sumamos los cuadrados de sus dígitos y seguimos el proceso con los resultados obtenidos, finalmente

obtenemos uno (1) como resultado. Por ejemplo, el número 203 es un número feliz ya que $2^2 + 0^2 + 3^2 = 13 \rightarrow 1^2 + 3^2 = 10 \rightarrow 1^2 + 0^2 = 1$.

Se dice que un número es feliz de grado k si se ha podido demostrar que es feliz en un máximo de k iteraciones. Se entiende que una iteración se produce cada vez que se elevan al cuadrado los dígitos del valor actual y se suman. En el ejemplo anterior, 203 es un número feliz para cualquier grado mayor o igual que 3.

En general, si un número es feliz de grado k' también es feliz para cualquier grado $k \geq k'$

Escriba un programa que lea un valor entero n y un valor k. Si el número es feliz para un grado k' menor o igual que k, el programa parará y mostrará dicho grado k'. En caso contrario, el programa nos dirá que no es feliz para cualquier grado menor o igual que k.

▷ Ejemplo de entrada: `13 2`
▷ Salida correcta: `Es feliz para cualquier grado >= 2`
▷ Ejemplo de entrada: `13 5`
▷ Salida correcta: `Es feliz para cualquier grado >= 2`
▷ Ejemplo de entrada: `13 1`
▷ Salida correcta: `No es feliz para cualquier grado <= 1`
▷ Ejemplo de entrada: `203 1`
▷ Salida correcta: `No es feliz para cualquier grado <= 1`
▷ Ejemplo de entrada: `203 7`
▷ Salida correcta: `Es feliz para cualquier grado >= 3`
▷ Ejemplo de entrada: `102 6`
▷ Salida correcta: `No es feliz para cualquier grado <= 6`

Finalidad: Ejercitar los bucles anidados. Dificultad Media.

66. [Gaussiana con un menú] Recupere la solución del ejercicio I.18 (función gaussiana) de la relación de problemas I. Se pide crear un menú principal para que el usuario pueda elegir las siguientes opciones:

```
Introducir parámetros de la función (esperanza y desviación)
Salir del programa
```

Si el usuario elige la opción de salir, el programa terminará; si elige la opción de introducir los parámetros, el programa leerá los dos parámetros (esperanza y desviación). La media puede ser un valor cualquiera, pero la desviación ha de ser un número positivo. A continuación, el programa presentará un menú con las siguientes opciones:

```
Introducir rango de valores de abscisas
Volver al menú anterior (el menú principal)
```

Si el usuario elige volver al menú anterior, el programa debe presentar el primer menú (el de la introducción de los parámetros) Si el usuario elige introducir los valores de abscisas, el programa le pedirá un valor `minimo`, un valor `maximo` (ha de ser mayor que `minimo`) y un `incremento` y mostrará el valor de la función gaussiana en todos los valores

de x (la abscisa) entre `minimo` y `maximo` a saltos de `incremento`, es decir, `minimo`, `minimo + incremento`, `minimo + 2*incremento`, $\cdots$, hasta llegar, como mucho, a `maximo`. Después de mostrar los valores de la función, el programa volverá al menú de introducción del rango de valores de abscisas.

▷ Ejemplo de entrada: `P 12 5 R 11 13 0.5 V S` (Se han elegido las letras `P` para introducir parámetros, `R` para introducir el rango, `V` para volver del menú secundario al principal y `S` para salir del programa)

▷ Salida correcta:

```
f(11)=0.0782085
f(11.5)=0.0793905
f(12)=0.0797885
f(12.5)=0.0793905
f(13)=0.0782085
```

Finalidad: Ejercitar los bucles anidados. Dificultad Media.

67. [Bits to char] *(Examen Julio 2017)* Implemente un programa que lea *bits* (ceros y unos) desde teclado, hasta que se introduzca un valor negativo. Si se introduce un positivo distinto de 0 y 1, el programa lo descartará y volverá a leer un valor.

    Cada 8 valores de *bits* leídos, el programa calculará el número entero que representa y lo transformará en el carácter (`char`) correspondiente. Debe tener en cuenta que el primer *bit* leído es el más significativo.

    Por ejemplo, si el usuario introduce los siguientes enteros:

    `0 1 0 0 1 3 4 1 1 1`

    acaba de completar un *octeto* ya que el 3 y el 4 no son bits (0 o 1) y se descartan. Por tanto, el octeto completado es `0 1 0 0 1 1 1 1` que corresponde al número entero 79 y por tanto al carácter `'O'`. El programa debe ir construyendo el entero 79 como suma de potencias de 2 ($0 * 2^7 + 1 * 2^6 + \cdots$)

    Si no es posible completar el último bloque con 8 bits (porque se haya introducido un negativo antes del octavo bit), se descartarán todos los bits de ese último bloque incompleto.

    Si el carácter obtenido corresponde a una letra -mayúscula o minúscula- lo mostrará por pantalla. Una vez terminada la entrada de datos, el programa mostrará el porcentaje de letras y otros símbolos (distintos de letras) leídos.

    ▷ Ejemplo de entrada:

    `0 1 0 0 1 3 4 1 1 1 2 0 1 1 0 1 0 1 1 9 0 0 1 7 0 0 0 0 1 1 0 -1`

    ▷ Salida correcta:

    ```
    Ok
    Letras: 66.67%
    Otros:  33.33%
    ```

Donde las correspondencias son $01001111 \rightarrow$ `O`, $01101011 \rightarrow$ `k`, y $00100001 \rightarrow$ `!`. Los *bits* finales 10 se descartan ya que no se ha completado un bloque de ocho.

Finalidad: Bucles anidados y lectura de datos. *Dificultad Media.*

68. Realice un programa que lea dos secuencias de enteros desde teclado y nos diga si todos los valores de la primera secuencia son mayores que todos los valores de la segunda secuencia.

    Realice la lectura de los enteros dentro de sendos bucles sobre una única variable llamada `dato`. El final de cada secuencia viene marcado cuando se lee el 0.

    Finalidad: Ejercitar el uso de bucles. *Dificultad Baja.*

69. Amplie el ejercicio I.38 (Población) de la relación de problemas I.

    Esta nueva versión del programa, además de los datos ya pedidos en dicho ejercicio, se le pedirá al usuario que introduzca un número de años (será el último dato leído) Debe leer cada dato con un filtro conveniente. Por ejemplo, las tasas de natalidad, mortalidad y emigración deben ser enteros entre 0 y 1000, mientras que la población inicial debe ser un entero positivo.

    El programa debe calcular e imprimir el número total de habitantes transcurridos dichos años.

    Además, el programa también calculará el número de años que tienen que pasar hasta que haya, como mínimo, el doble de la población inicial. Imprima dicho número de años, junto con la población que habrá pasado ese tiempo.

    Por ejemplo, para la siguiente entrada

    ```
    1375570814  <- Población inicial
    32          <- Tasa de natalidad
    12          <- Tasa de mortalidad
    7           <- Tasa de migración
    3           <- Número de años
    ```

    el programa debe devolver lo siguiente:

    ```
    1490027497  <- Número de habitantes pasados 3 años
    27          <- Años que han de pasar hasta doblar la población
    2824131580  <- Población transcurridos 27 años
    ```

    ▷ Ejemplo de entrada: `1375570814  32  12  7  3`
    ▷ Salida correcta: `1490027497  27  2824131580`

    Finalidad: Ejemplo básico de asignación acumulada. *Dificultad Baja.*

70. Una empresa que tiene tres sucursales decide llevar la contabilidad de las ventas de sus productos a lo largo de una semana. Para ello registra cada venta con tres números, el identificador de la sucursal (1, 2 o 3), el código del producto codificado como un carácter (a, b ó c) y el número de unidades vendidas. Diseñar un programa que lea desde el

teclado una serie de registros compuestos por `sucursal, producto, unidades` y diga cuál es la sucursal que más productos ha vendido. La serie de datos termina cuando la sucursal introducida vale -1. Por ejemplo, con la serie de datos

```
2 a 20
1 b 10
1 b 4
3 c 40
1 a 1
2 b 15
1 a 1
1 c 2
2 b 6
-1
```

Se puede ver que la sucursal que más productos ha vendido es la número 2 con 41 unidades totales. Para comprobar que el programa funciona correctamente, cread un fichero de texto y re-dirigid la entrada a dicho fichero.

Finalidad: Ver un bucle en el que se leen varios datos en cada iteración, pero sólo uno de ellos se usa como terminador de la entrada. Dificultad Media.

71. Se quiere construir un programa para leer los datos necesarios del ejercicio II.57 de la subida salarial.

Supondremos que sólo hay tres empleados y que están identificados con un código (1, 2 y 3). Además, el salario por hora es el mismo para todos los empleados. Éste será el primer valor que se leerá (de tipo `double`) Después de haber leído este dato, se leerán los datos de los casos atendidos por los empleados en el siguiente orden: en primer lugar, el código del empleado, a continuación el número de segundos que ha durado la atención telefónica, en tercer lugar un 1 si el caso se resolvió de forma satisfactoria y un 0 en caso contrario; finalmente, un valor entero entre 0 y 5 con el grado de satisfacción del usuario.

Cuando nos encontremos el terminador `-1` como primer dato (código del empleado) se detendrá la introducción de datos. Supondremos que siempre se introduce al menos el primer valor (el salario), pudiendo ser ya el siguiente dato leído el terminador.

```
7.5         <- Salario de 7.5 euros por hora
2 124 1 3   <- Empleado 2, 124", resuelto,    grado sat: 3
1 32 0 0    <- Empleado 1, 32" , no resuelto, grado sat: 0
2 26 0 2    <- Empleado 2, 26" , no resuelto, grado sat: 2
-1          <- Fin de entrada de datos
```

El programa debe imprimir el número total de casos introducidos (3 en el ejemplo anterior) y el código del empleado con mayor grado de satisfacción medio (también imprimirá dicho grado medio). En el ejemplo anterior, sería el empleado 2 con un nivel medio de

satisfacción de `2.5` (observe que el grado medio se calcula en relación al número total de casos atendidos y no sobre los casos resueltos).

Observe que, en este ejercicio, no se están teniendo en cuenta los datos referentes al tiempo de cada caso y si fue resuelto o no, pero hay que leer todos los datos para llegar a los que sí nos interesan.

▷ Ejemplo de entrada: `7.5  2 124 1 3  1 32 0 0  2 26 0 2  -1`
▷ Salida correcta: `3  2  2.5`

▷ Ejemplo de entrada: `7.5 -1`
▷ Salida correcta: `No se introdujo ningún caso`

Finalidad: Plantear un bucle de lectura de datos. Dificultad Baja.

72. [Número desgarrable] Un número entero `n` se dice que es *desgarrable* si al dividirlo en dos partes cualesquiera `izda` y `dcha`, el cuadrado de la suma de ambas partes es igual a `n`. Por ejemplo, 88209 es desgarrable ya que $(88 + 209)^2 = 88209$; 81 también lo es ya que $81 = (8 + 1)^2$. Cree un programa que lea un entero `n` e indique si es o no desgarrable.

    Finalidad: Ejercitar los bucles con condiciones de salida complejas. Dificultad Media.

73. [RLE] El método RLE (Run Length Encoding) codifica una secuencia de datos formada por series de valores idénticos consecutivos como una secuencia de parejas de números (valor de la secuencia y número de veces que se repite). Esta codificación es un mecanismo de compresión de datos (zip) sin pérdidas. Se aplica, por ejemplo, para comprimir los ficheros de imágenes en las que hay zonas con los mismos datos (fondo blanco, por ejemplo). Realice un programa que lea una secuencia de números naturales terminada con un número negativo y la codifique mediante el método RLE.

Entrada:	1 1 1 2 2 2 2 2 3 3 3 3 3 3 5 -1
	(tres veces 1, cinco veces 2, seis veces 3, una vez 5)
Salida:	3 1 5 2 6 3 1 5

    Finalidad: Controlar en una iteración lo que ha pasado en la anterior. Dificultad Media.

74. [Pinta dígitos arbitrario] En el ejercicio I.37 [Pinta dígitos] de la Relación de Problemas I se pedía escribir un programa que leyese un valor entero de tres dígitos e imprimiese los dígitos separados por un espacio en blanco. Haga lo mismo pero para un número entero arbitrario. Por ejemplo, si el número es 3519, la salida sería:

    ```
    3   5   1   9
    ```

    En este ejercicio se pueden mezclar entradas y salidas con cómputos.

    Finalidad: Trabajar con bucles que recorren los dígitos de un número. Dificultad Media.

75. El algoritmo de la multiplicación rusa es una forma distinta de calcular la multiplicación de dos números enteros $n * m$. Para ello este algoritmo va calculando el doble del

multiplicador m y la mitad (sin decimales) del multiplicando n hasta que n tome el valor 1 y suma todos aquellos multiplicadores cuyos multiplicandos sean impares. Por ejemplo, para multiplicar 37 y 12 se harían las siguientes iteraciones

Iteración	Multiplicando	Multiplicador
1	**37**	12
2	18	24
3	**9**	48
4	4	96
5	2	192
6	**1**	384

Con lo que el resultado de multiplicar 37 y 12 sería la suma de los multiplicadores correspondientes a los multiplicandos impares (en negrita), es decir 37*12=12+48+384=444

Cree un programa para leer dos enteros n y m y calcule su producto utilizando este algoritmo. No puede utilizarse en ningún momento el operador producto `*`.

Dificultad Media.

76. Amplíe el ejercicio II.3 (año bisiesto). El programa pedirá los valores de dos años obligando a que estén entre el año cero y 2100. A continuación, el programa mostrará todos los años bisiestos comprendidos entre los años anteriores.

    Finalidad: Practicar con filtros y ciclos básicos.Practicar con algoritmos más elaborados y eficientes. Reutilizar código ya escrito y verificado. Dificultad Media.

77. Todo lo que se puede hacer con un bucle `while` se puede hacer con un `do while`. Lo mismo ocurre al revés. Sin embargo, cada bucle se usa de forma natural en ciertas situaciones. El no hacerlo, nos obligará a escribir más código y éste será más difícil de entender. Para comprobarlo, haced lo siguiente:

    1. Modifique la solución del ejercicio II.18 de forma que el filtro de entrada usado para leer la variable `tope`, se haga con un bucle pre-test `while`.
    2. Modifique la solución del ejercicio II.23 sustituyendo el bucle `while` por un `do while`. Observad que debemos considerar el caso en el que el número de años leído fuese cero.

    Finalidad: Enfatizar la necesidad de saber elegir entre un bucle pre-test o un bucle post-test. Dificultad Media.

78. Calcule mediante un programa en C++ el combinatorio $\binom{n}{m}$ con n, m valores enteros.

    No pueden usarse las funciones de la biblioteca `cmath`.

    El combinatorio de n sobre m (con $n \geq m$) es un número entero que se define como

sigue:

$$\left(\begin{array}{c} n \\ m \end{array} \right) = \frac{n!}{m!\,(n-m)!}$$

Finalidad: Trabajar con bucles controlados por contador. Dificultad Media.

79. Construya un programa que lea un valor T y calcule la siguiente sumatoria:

$$\sum_{i=1}^{i=T} i! = \sum_{i=1}^{i=T} \left(\prod_{j=1}^{j=i} j \right)$$

Por ejemplo, para $T = 4$, la operación a realizar es:

$$1! + 2! + 3! + 4!$$

es decir:

$$1 + (1 * 2) + (1 * 2 * 3) + (1 * 2 * 3 * 4)$$

▷ Ejemplo de entrada: 3 ▷ Salida correcta: 9
▷ Ejemplo de entrada: 4 ▷ Salida correcta: 33
▷ Ejemplo de entrada: 6 ▷ Salida correcta: 873

Finalidad: Ejercitar los bucles anidados. Dificultad Media.

80. Resuelva el ejercicio II.79 sin utilizar bucles anidados, es decir, debe usar un único bucle.

Finalidad: Aprovechar en una iteración los cómputos hechos en la iteración anterior. Dificultad Media.

81. Diremos que un número entero positivo es secuenciable si se puede generar como suma de números consecutivos (al menos dos). Por ejemplo, $6 = 1 + 2 + 3$, $15 = 7 + 8$. Esta descomposición no tiene por qué ser única. Por ejemplo, $15 = 7 + 8 = 4 + 5 + 6 = 1 + 2 + 3 + 4 + 5$. Escriba un programa que lea un entero $n \geq 1$ e imprima todas las descomposiciones posibles. En este ejercicio puede mezclar operaciones de E/S y C dentro del mismo bucle.

Como curiosidad, los únicos números con 0 descomposiciones son las potencias de 2.

▷ Ejemplo de entrada: 6 ▷ Salida correcta: `1 2 3`
▷ Ejemplo de entrada: 15 ▷ Salida correcta: `7 8 / 4 5 6 / 1 2 3 4 5`

Finalidad: Ejercitar los bucles anidados. Dificultad Media.

82. *(Examen Septiembre 2014)* ¿Cuántas veces aparece el dígito 9 en todos los números que hay entre el 1 y el 100? Por ejemplo, el 9 aparece una vez en los números 19 y 92 mientras que aparece dos veces en el 99. Pretendemos diseñar un algoritmo que responda a esta sencilla pregunta, pero de forma suficientemente generalizada. Para

ello, se pide construir un programa que lea una `cifra` (entre 1 y 9), dos enteros `min` y `max` y calcule el número de apariciones del dígito `cifra` en los números contenidos en el intervalo cerrado `[min, max]`.

Finalidad: Ejercitar los bucles anidados. Dificultad Baja.

83. [Progresión geométrica] Supongamos una serie numérica cuyo término general es:

$$a_i = a_1 r^{i-1}$$

Es decir, la serie la forman los siguientes términos:

$a_1 = a_1$
$a_2 = a_1 r$
$a_3 = a_1 r^2$
$a_4 = a_1 r^3$
$\dots$

Se pide crear un programa que lea desde teclado r, el primer elemento a_1 y el tope k y calcule la suma de los primeros k valores de la serie, es decir:

$$\sum_{i=1}^{i=k} a_i$$

Se proponen dos alternativas:

1. Realice la suma de la serie usando la función `pow` para el cómputo de cada término a_i. Los argumentos de `pow` no pueden ser ambos enteros, por lo que forzaremos a que la base (por ejemplo) sea `double`, multiplicando por `1.0`.
2. Si analizamos la expresión algebraica de la serie numérica, nos damos cuenta que es una *progresión geométrica* ya que cada término de la serie queda definido por la siguiente expresión:

   $$a_{i+1} = a_i * r$$

   Es decir, una progresión geométrica es una secuencia de elementos en la que cada uno de ellos se obtiene multiplicando el anterior por una constante denominada razón o factor de la progresión.

   Cree el programa pedido usando esta fórmula. NO puede utilizarse la función `pow`.

¿Qué solución es preferible en términos de eficiencia?

Finalidad: Trabajar con bucles que aprovechan cómputos realizados en la iteración anterior. Dificultad Baja.

84. Reescribid la solución a los ejercicios II.18 (divisores) y II.23 (interés) usando un bucle `for`

Finalidad: Familiarizarnos con la sintaxis de los bucles `for`. *Dificultad Baja.*

85. Diseñar un programa para calcular la suma de los 100 primeros términos de la sucesión siguiente:

$$a_i = \frac{(-1)^i(i^2-1)}{2i}$$

No puede usarse la función `pow`. Hacedlo calculando explícitamente, en cada iteración, el valor $(-1)^i$ (usad un bucle `for`). Posteriormente, resolvedlo calculando dicho valor a partir del calculado en la iteración anterior, es decir, $(-1)^{i-1}$.

Finalidad: Enfatizar la conveniencia de aprovechar cómputos realizados en la iteración anterior. Dificultad Media.

86. Se pide diseñar un programa para jugar a adivinar un número entre 1 y 100. El juego tiene que dar pistas de si el número introducido por el jugador está por encima o por debajo del número introducido. Como reglas de parada se consideran los siguientes dos casos:
a) se ha acertado b) se decide abandonar el juego (decida cómo quiere especificar esta opción)

Para poder generar números aleatorios en un rango determinado será necesario incluir las siguientes instrucciones:

```
#include <iostream>
#include <ctime>
#include <cstdlib>
using namespace std;

int main(){
   const int MIN = 1,MAX = 100;
   const NUM_VALORES = MAX-MIN + 1;        // rango
   int incognita;                          // número generado
   time_t tiempo;

   // Inicialización de la secuencia:
   srand(time(&tiempo));

   // Generación de un número aleatorio incognita:
   // MIN <= incognita <= MAX
   incognita = (rand() \% NUM_VALORES) + MIN;
```

La sentencia `srand(time(&tiempo))` debe ejecutarse una única vez al principio del programa y sirve para inicializar la secuencia de números aleatorios. Posteriormente, cada vez que se ejecute la sentencia `incognita = (rand() \% NUM\_VALORES) + MIN;` se obtendrá un valor aleatorio (pseudoaleatorio).

Realizar el mismo ejercicio pero permitiendo jugar tantas veces como lo desee el jugador.

Dificultad Media.

87. Implemente un programa que sea capaz de "dibujar" rectángulos utilizando un símbolo (un carácter) dado. El usuario ingresará el símbolo $simb$, la altura M y el ancho N del rectángulo. Por ejemplo, siendo $simb =*$, $M = 3$ y $N = 5$, el dibujo tendría la siguiente forma:

```
*****
*****
*****
```

Finalidad: Ejercitar los bucles anidados. *Dificultad Baja.*

88. Implemente un programa que sea capaz de "dibujar" pinos utilizando asteriscos "*". El usuario ingresara el ancho de la base del pino (podemos asumir que es un número impar). Supongamos que se ingresa 7, entonces el dibujo tendrá la siguiente forma:

```
   *
  ***
 *****
*******
  ***
  ***
```

Finalidad: Ejercitar los bucles anidados. *Dificultad Media.*

89. Realizar un programa para calcular los valores de la función:

$$f(x) = \sqrt{\frac{3x + x^2}{1 - x^2}}$$

para valores de x enteros en el rango [-3..3].
Dificultad Baja.

90. Realizar un programa para calcular los valores de la función:

$$f(x, y) = \frac{\sqrt{x}}{y^2 - 1}$$

para los valores de (x, y) con $x = -50, -48, \ldots, 48, 50 \quad y = -40, -39, \ldots, 39, 40$, es decir queremos mostrar en pantalla los valores de la función en los puntos

$$(-50, 40), (-50, -39), \cdots (-50, 40), (-48, 40), (-48, -39), \cdots (50, 40)$$

Dificultad Baja.

91. Diseñar un programa que presente una tabla de grados C a grados Fahrenheit (F=9/5C+32) desde los 0 grados a los 300, con incremento de 20 en 20 grados.
    Dificultad Baja.

92. Diseñar un programa que lea caracteres desde la entrada y los muestre en pantalla, hasta que se pulsa el `'.'` y diga cuántos separadores se han leído (espacios en blanco `' '`, tabuladores `'\t'` y caracteres de nueva línea `'\n'`).
    Dificultad Baja.

93. Realizar un programa para calcular la suma de los términos de la serie
    $$1 - 1/2 + 1/4 - 1/6 + 1/8 - 1/10 + ... - 1/(2n-1) + 1/(2n)$$
    para un valor n dado.
    Dificultad Baja.

94. Se decide informatizar el acta de un partido de baloncesto para saber qué equipo es el ganador del partido. El acta contiene una serie de anotaciones formadas por una pareja de números cada una, con el dorsal del jugador y el número de puntos conseguidos teniendo en cuenta que la última anotación es un valor -1. Por ejemplo
    1 2 4 1 4 1 2 3 6 2 3 2 5 2 5 1 1 3 -1
    El programa deberá indicar si ha ganado el equipo 1 (con los dorsales 1, 2 y 3) o el equipo 2 (dorsales 4, 5 y 6) o han empatado.
    Por ejemplo, con la entrada anterior, gana el equipo 1.
    Dificultad Baja.

95. La Unión Europea ha decidido premiar al país que más toneladas de hortalizas exporte a lo largo del año. Se dispone de un registro de transacciones comerciales en el que aparecen tres valores en cada apunte. El primer valor es el indicativo del país (E: España, F: Francia y A: Alemania), el segundo valor es un indicativo de la hortaliza que se ha vendido en una transacción (T: Tomate, P: Patata, E: Espinaca) y el tercer valor indica las toneladas que se han vendido en esa transacción. Diseñar un programa que lea desde el teclado este registro, el cual termina siempre al leer un país con indicativo '@', y que diga qué país es el que más hortalizas exporta y las toneladas que exporta.
    Por ejemplo, con la entrada
    E T 10 E T 4 E P 1 E P 1 E E 2 F T 15 F T 6 F P 20 A E 40 @
    el país que más vende es Francia con un total de 41 toneladas.
    Dificultad Baja.

96. Se pide leer dos enteros sabiendo que el primero no tiene un tamaño fijo y que el segundo siempre es un entero de dos dígitos. Se pide comprobar si el segundo está contenido en el primero. Entendemos que está contenido si los dos dígitos del segundo entero están en el primer entero de forma consecutiva y en el mismo orden. Por ejemplo, 89 está contenido en 7890, en 7789 y en 8977 pero no en 7980.

Dificultad Media.

97. Se dice que un número es triangular si se puede poner como la suma de los primeros m valores enteros, para algún valor de m. Por ejemplo, 6 es triangular ya que 6 = 1 + 2 + 3. Se pide construir un programa que obtenga todos los números triangulares que hay menores que un entero `tope` introducido desde teclado.

    Dificultad Baja.

98. Escriba un programa que lea por teclado un número entero positivo tope y muestre por pantalla el factorial de los tope primeros números enteros. Recuerda que el factorial de un número entero positivo n es igual al producto de los enteros positivos del 1 al n.

    Dificultad Baja.

99. Construya un programa para comprobar si las letras de una palabra se encuentran dentro de otro conjunto de palabras. Los datos se leen desde un fichero de la forma siguiente: el fichero contiene, en primer lugar un total de 3 letras que forman la palabra a buscar, por ejemplo `f e o`. Siempre habrá, exactamente, tres letras. A continuación, el fichero contiene el conjunto de palabras en el que vamos a buscar. El final de cada palabra viene determinado por la aparición del carácter '`@`', y el final del fichero por el carácter '`#`'. La búsqueda tendrá las siguientes restricciones:

    - Deben encontrarse las tres letras
    - Debe respetarse el orden de aparición. Es decir, si por ejemplo encontramos la '`f`' en la segunda palabra, la siguiente letra a buscar '`e`' debe estar en una palabra posterior a la segunda.
    - Una vez encontremos una letra en una palabra, ya no buscaremos más letras en dicha palabra.
    - No nos planteamos una búsqueda barajando todas las posibilidades, en el sentido de que una vez encontrada una letra, no volveremos a buscarla de nuevo.

Entrada:	`f e o`
	`h o l a @`
	`m o f e t a @  <- f`
	`c o f i a @`
	`c e r r o @  <- e`
	`p e r a @`
	`c o s a @  <- o`
	`h o y @`
	`#`

En este caso, sí se encuentra.

Dificultad Media.

100. Un número perfecto es aquel que es igual a la suma de todos sus divisores positivos excepto él mismo. El primer número perfecto es el 6 ya que sus divisores son 1, 2 y 3 y 6=1+2+3. Escribir un programa que muestre el mayor número perfecto que sea menor a un número dado por el usuario.

Dificultad Media.

101. Escribir un programa que encuentre dos enteros n y m mayores que 1 que verifiquen lo siguiente:

$$\sum_{i=1}^{m} i^2 = n^2$$

Dificultad Media.

102. En matemáticas, la **sucesión de Fibonacci** (a veces mal llamada *serie* de Fibonacci) es la siguiente sucesión infinita de números naturales:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$$

La sucesión comienza con los números 1 y 1, y a partir de éstos, cada término puede calcularse como la suma de los dos anteriores. A los elementos de esta sucesión se les llama *números de Fibonacci*.

El número de Fibonacci de orden n, al que llamaremos f_n se puede definir mediante la siguiente relación de recurrencia:

- $f_n = f_{n-1} + f_{n-2}$ para $n > 2$
- $f_1 = f_2 = 1$

Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemáticas y teoría de juegos. También aparece en diversas configuraciones biológicas.

Escribir un programa que calcule el número de Fibonacci de orden n, donde n es un valor introducido por el usuario. A continuación, el programa solicitará un nuevo valor, k, y mostrará todos los números de Fibonacci $f_1, f_2, f_3, \ldots, f_k$.

Finalidad: Trabajar con bucles controlados por contador. *Dificultad Media.*

103. El número aúreo se conoce desde la Antigüedad griega y aparece en muchos temas de la geometría clásica. La forma más sencilla de definirlo es como el único número positivo ϕ que cumple que $\phi^2 - \phi = 1$ y por consiguiente su valor es $\phi = \dfrac{1+\sqrt{5}}{2}$.

Se pueden construir aproximaciones al número aúreo mediante la fórmula $a_n = \dfrac{f_{n+1}}{f_n}$ siendo f_n el número de Fibonacci de orden n (ver problema II.102).

La sucesión de valores así calculada proporciona, alternativamente, valores superiores e inferiores a ϕ, siendo cada vez más cercanos a éste, y por lo tanto la diferencia entre a_n y ϕ es cada vez más pequeña conforme n se hace mayor.

Escribir un programa que calcule el menor valor de n que hace que la aproximación dada por a_n difiera en menos de δ del número ϕ, sabiendo que $n \geq 1$.

La entrada del programa será el valor de δ y la salida el valor de n. Por ejemplo, para un valor de $\delta = 0.1$ el valor de salida es $n = 4$

Dificultad Media.

104. Una *sucesión alícuota* es una sucesión iterativa en la que cada término es la suma de los divisores propios del término anterior. La sucesión alícuota que comienza con el entero positivo k puede ser definida formalmente mediante la función divisor σ_1 de la siguiente manera:

$$s_0 = k$$
$$s_n = \sigma_1(s_{n-1}) - s_{n-1}$$

Por ejemplo, la sucesión alícuota de 10 es $10, 8, 7, 1, 0$ porque:

$$\sigma_1(10) - 10 = 5 + 2 + 1 = 8$$
$$\sigma_1(8) - 8 = 4 + 2 + 1 = 7$$
$$\sigma_1(7) - 7 = 1$$
$$\sigma_1(1) - 1 = 0$$

Aunque muchas sucesiones alícuotas terminan en cero, otras pueden no terminar y producir una sucesión alícuota períodica de período 1, 2 o más. Está demostrado que si en una sucesión alícuota aparece un *número perfecto* (como el 6) se produce una sucesión infinita de período 1. Un *número amigable* produce una sucesión infinita de período 2 (como el 220 ó 284).

Escribir un programa que lea un número natural menor que 1000 y muestre su sucesión alícuota. Hay que tener en cuenta que en ocasiones se pueden producir sucesiones infinitas, por lo que en estos casos habrá que detectarlas e imprimir puntos suspensivos cuando el período se repita. Solo hay que considerar períodos infinitos de longitud 2 como máximo. Por ejemplo; para el número 6, se imprimiría: $6, 6, \ldots$; y para el número 220, se imprimiría: $220, 284, 220, 284, \ldots$.

Finalidad: Practicar los bucles anidados y controlar las condiciones de parada a partir de lo sucedido en iteracionse anteriores. Dificultad Media.

105. [Cuadro de límites de velocidad] La Dirección General de Tráfico publica en su página web la sanción a aplicar cuando se sobrepasa el límite de velocidad establecido en una

vía. La sanción consta de una multa económica y una detracción del número de puntos del carnet del infractor.

El siguiente cuadro está publicado en

https://sede.dgt.gob.es/Galerias/tramites-y-multas/alguna-multa/consulta-de-sanciones-por-exceso-velocidad/cuadro_velocidad.pdf

MINISTERIO DEL INTERIOR — Dirección Gral. de Tráfico

Cuadro para excesos de velocidad (Fecha de entrada en vigor 25/05/2010)

Límite		30	40	50	60	70	80	90	100	110	120	Multa	Puntos
Exceso de velocidad	Grave	31 50	41 60	51 70	61 90	71 100	81 110	91 120	101 130	111 140	121 150	100	-
		51 60	61 70	71 80	91 110	101 120	111 130	121 140	131 150	141 160	151 170	300	2
		61 70	71 80	81 90	111 120	121 130	131 140	141 150	151 160	161 170	171 180	400	4
		71 80	81 90	91 100	121 130	131 140	141 150	151 160	161 170	171 180	181 190	500	6
	Muy Grave	81	91	101	131	141	151	161	171	181	191	600	6

Escriba un programa en C++ para imprimir los límites inferiores indicados dentro del cuadro. Es decir, el programa debe imprimir lo siguiente:

```
31 41 51  61  71  81  91  101 111 121
51 61 71  91  101 111 121 131 141 151
61 71 81  111 121 131 141 151 161 171
71 81 91  121 131 141 151 161 171 181
81 91 101 131 141 151 161 171 181 191
```

Debe usar un bucle anidado en otro y establecer los incrementos convenientes. Por ejemplo, el incremento entre dos columnas consecutivas de una misma fila es de 10, salvo el incremento que hay entre la tercera y cuarta columnas que es de 10 para la primera fila, de 20 para la segunda y de 30 para la tercera fila y siguientes.

Finalidad: Ejercitar los bucles anidados. Dificultad Media.